survlearning: A Unified R Interface for Survival Learning with Censored Data

Pedro Luiz Ramos
Pontificia Universidad Católica de Chile

Abstract

In this paper, we present **survlearning**, a formula-based R interface for fitting, tuning, and comparing established survival learning methods. The package separates risk scores, survival probabilities, and time predictions. Data preprocessing and censoring models are estimated using only the training observations in each fold. We consider 25 configurations covering accelerated failure time and Cox models, regularization, trees, forests, boosting, multi-task logistic regression, neural networks, and support vector machines. For SVMs, we consider censored regression, ranking, a hybrid loss, and inverse-probability-weighted regression with linear or additive kernels. Sparse representations and analytic gradients allow estimation without removing observations or approximating the selected kernel. Neural Cox estimation is implemented in R and accounts for tied failures without Torch or Python. The package also provides interpretation, conditional reliability summaries, paired comparison intervals, censoring-weighted calibration measures, and convex survival stacking. A generated example includes all 25 configurations. An application compares 20 configurations and a survival stack using all training observations from 29,058 eligible hard drives with 718 failures. Additive ranking and hybrid SVMs obtain test concordance 0.833, compared with 0.856 for random survival forests and 0.695 for additive censored regression. These results concern this historical data split and do not establish a general ordering of the methods. Independent calculations check numerical accuracy, and separate processes measure time and memory. The package uses the GNU General Public License, version 3 or later.

Keywords: survival analysis, reliability, statistical learning, censored data, regularization, concordance, R.

1. Introduction

Reliability studies involve components operating under different conditions. For hard drives, manufacturer, capacity, operating hours, temperature and diagnostic measurements can help predict failures. A risk score can identify drives needing attention, whereas replacement planning may require failure probabilities or remaining-time summaries. Since many drives remain operating when observation ends, estimation must account for right censoring.

Classical models provide a formulation for this problem (Lawless 2003; Kalbfleisch and Prentice 2002). Accelerated failure time (AFT) models relate covariates to the time scale, whereas Cox regression models the hazard (Cox 1972). Regularization, trees, boosting and kernels extend the available fits. Their assumptions and prediction types differ; a common software interface should make these differences clear.

Established R engines (R Core Team 2026) include **survival**, **glmnet**, **penAFT**, **rpart**, **randomForestSRC**, **gbm**, **survivalsvm**, **xgboost**, and **MTLR**. Combining them requires consistent predictor coding, objective-specific penalties, a common risk-score direction, and parameter selection separate from test evaluation.

Unified survival learning is not a new software objective. The **mlr3proba** framework provides survival tasks, learners, prediction objects, and evaluation within the **mlr3** ecosystem (Sonabend, Király, Bender, Bischl, and Lang 2021). The **censored** package extends the **parsnip** interface to censored regression (Hvitfeldt and Frick 2026). More recent packages include **survalis**, with modeling and interpretation facilities, and **SuperSurv**, with a cross-validated ensemble and best-learner selection modes (El Badisy 2026; Lyu 2026; SuperSurv Authors 2026). **survcompare** focuses on repeated nested comparisons of Cox, Cox-LASSO, RSF and their combinations (Shamsutdinova and Stahl 2025). In Python, **scikit-survival** provides an interface compatible with **scikit-learn** (Pölsterl 2020). These projects also include features outside the scope considered here. Section 8 compares matched fits with **censored**.

In this paper, we present **survlearning**, organized around `surv_fit()`, `surv_tune()`, `surv_predict()` and `surv_score()`. We simplify fitting and evaluation while preserving each method's meaning. Neural Cox handles ties in R without an external runtime. SVM estimation uses the primal structure without approximating kernels or removing observations. Censored regression, ranking, hybrid and weighted losses remain distinct from kernel choice. No new loss is proposed. Incorporated code retains its authorship and licenses.

Larger scores always indicate greater risk; preprocessing uses training observations only. Unsupported probabilities or time predictions are not invented, including for SVM scores.

Sections 2–4 describe the prediction targets, estimation objectives, and interface. Sections 5–7 present the examples and post-fit analyses. Numerical verification, replication, and limitations follow in Sections 8–10.

2. Observed data and prediction targets

2.1. Right censoring and the survival function

Let $T_i > 0$ denote the failure time, $C_i > 0$ the censoring time, and $x_i \in \mathbb{R}^p$ the covariate vector for unit i. We observe

$$Y_i = \min(T_i, C_i), \quad \delta_i = \mathbb{I}(T_i \le C_i), \quad \mathcal{D} = \{(Y_i, \delta_i, x_i) : i = 1, \dots, n\}. \tag{1}$$

The usual likelihood assumes conditionally independent censoring, $T_i \perp C_i \mid x_i$, and independent units. Using a survival learning method does not ensure that these assumptions hold. For example, removing a drive before failure may be related to its condition and hence informative. The conditional survival, hazard, and cumulative hazard functions are given by

$$\begin{aligned} S(t \mid x) &= \Pr(T > t \mid x), \\ h(t \mid x) &= \lim_{a \downarrow 0} \frac{\Pr(t \le T < t + a \mid T \ge t, x)}{a}, \qquad H(t \mid x) = \int_0^t h(u \mid x)\, du. \end{aligned} \tag{2}$$

For a continuous failure time, $S(t \mid x) = \exp\{-H(t \mid x)\}$. Ignoring the censoring distribution under the stated assumption, the likelihood contribution is

$$L_i = f(Y_i \mid x_i)^{\delta_i} S(Y_i \mid x_i)^{1-\delta_i}. \tag{3}$$

Thus, a censored observation contributes a survival probability, not a density at an observed failure time. Replacing every censored time by an event time does not preserve Equation 3.

For descriptive analysis, let $t_1 < \cdots < t_J$ be the distinct observed event times, d_j the number of events at t_j, and n_j the number at risk just before t_j. The Kaplan–Meier estimator is

$$\widehat{S}_{\mathrm{KM}}(t) = \prod_{j:t_j \le t} \left(1 - \frac{d_j}{n_j}\right). \tag{4}$$

Equation 4 describes failure behavior within covariate groups (Kaplan and Meier 1958), not causal effects or individual prediction calibration.

2.2. Risk scores, probabilities, and time summaries

A score $r(x_a) > r(x_b)$ indicates greater predicted risk for unit a. Cox linear predictors, forest mortality and negative predicted times follow this convention on different numerical scales; they are not probabilities. When a survival function is available, the failure probability at horizon t_0 is $F(t_0 \mid x) = 1 - S(t_0 \mid x)$. The conditional median and restricted mean are given by

$$m(x) = \inf\{t : S(t \mid x) \le 1/2\}, \qquad \mu_\tau(x) = \int_0^\tau S(t \mid x)\, dt. \tag{5}$$

The horizon τ must be specified for the restricted mean. If an estimated curve remains above $1/2$ over the observed support, a median cannot be obtained without a tail assumption. A missing nonparametric median is not evidence of an infinite lifetime or a cured component. The current package does not fit cure models.

2.3. Harrell concordance

For distinct observed times, define the comparable ordered pairs by $\mathcal{P} = \{(i,j) : \delta_i = 1, Y_i < Y_j\}$. With a larger score indicating greater risk, Harrell's concordance index is estimated by

$$\widehat{C} = \frac{\displaystyle\sum_{(i,j)\in\mathcal{P}} \left[\mathbb{I}\{r(x_i) > r(x_j)\} + \tfrac{1}{2}\mathbb{I}\{r(x_i) = r(x_j)\}\right]}{|\mathcal{P}|}. \tag{6}$$

The formula assigns half credit to tied scores. When observed times are tied, the calculation uses $\mathcal{P}_H = \mathcal{P} \cup \{(i,j) : \delta_i = 1, \delta_j = 0, Y_i = Y_j\}$ in both sums. An event is treated as preceding a censoring recorded at the same time, whereas two events at the same time are not ordered. This follows `survival::concordance()`, with its numerical time-tie correction, rather than added numerical perturbations. The actual computation uses `reverse = TRUE`, since larger scores indicate shorter survival, and the default Harrell time weighting. Values near one indicate concordant ordering; values near $1/2$ indicate little ordering information (Harrell, Califf, Pryor, Lee, and Rosati 1982; Harrell, Lee, and Mark 1996). Harrell concordance depends on censoring and does not assess calibration. These examples select base models by Harrell concordance. Truncated Uno concordance, time-dependent AUC, Brier scores and calibration require their stated prediction types and censoring support.

3. Statistical learning methods

We first describe how each method is estimated. Although the functions have a common form, the fitted models are not interchangeable. The 25 identifiers considered here represent configurations within several families, rather than 25 unrelated algorithms. Table 1 links models, engines, and implemented prediction types. Code uses the data objects defined in the replication script and the examples below; the conditional-censoring example uses its separate generated cohort.

Identifier	Engine	Prediction types
`aft`	**survival**	Risk, median, survival
`aft_lasso`, `aft_ridge`	Integrated Gehan ADMM	Risk
`aft_elasticnet`	Integrated Gehan ADMM	Risk
`cox`	**survival**	Risk, survival
`cox_lasso`, `cox_ridge`	**glmnet**	Risk, survival
`cox_elasticnet`	**glmnet**	Risk, survival
`tree`	**rpart**	Risk
`rsf`	Integrated RF-SRC core	Risk, survival
`boosting`	**gbm**	Risk
`xgboost_cox`	**xgboost**	Risk, survival
`xgboost_aft`	**xgboost**	Risk, median, survival
`mtlr`	**MTLR**	Risk, survival
`svm`	Exact sparse primal	Risk
`svm_rank`	Native exact-pair optimization	Risk
`svm_hybrid`, `svm_ipcw`	Native analytic-gradient optimization	Risk
`cox_spline`	**survival**	Risk, survival
`flexsurv`	Integrated spline likelihood	Risk, survival
`coxboost`	**CoxBoost**	Risk, survival
`ranger`, `aorsf`	**ranger**, **aorsf**	Risk, survival
`neural_cox`	Native R	Risk, survival
`neural_discrete`	**nnet**	Risk, survival

Table 1: Covariate-based methods and prediction capabilities in version 0.3.5. Risk scores increase with risk, but their scales are not comparable across methods. An unavailable prediction type generates an error.

3.1. Parametric accelerated failure time models

The AFT formulation relates the logarithm of failure time to covariates:

$$\log T_i = \beta_0 + x_i^\top \beta + \sigma\varepsilon_i, \qquad \sigma > 0. \tag{7}$$

Let $\eta_i = \beta_0 + x_i^\top \beta$ and $z_i = (\log Y_i - \eta_i)/\sigma$. For error density f_ε and survival function S_ε, the log-likelihood is

$$\ell = \sum_{i=1}^{n} \left[\delta_i\{\log f_\varepsilon(z_i) - \log\sigma - \log Y_i\} + (1-\delta_i)\log S_\varepsilon(z_i)\right]. \tag{8}$$

The estimates are obtained by numerical maximization using `survival::survreg()` (Therneau 2026; Therneau and Grambsch 2000). Normal, logistic, and minimum extreme-value

errors give log-normal, log-logistic, and Weibull failure times, respectively. Under Equation 7, a one-unit increase in covariate j multiplies each conditional time quantile by $\exp(\beta_j)$, holding other covariates fixed. This time-ratio interpretation differs from a hazard ratio.

The fitted survival function is $\widehat{S}(t \mid x) = S_\varepsilon\{(\log t - \widehat{\eta}(x))/\widehat{\sigma}\}$. Its median is $\widehat{m}(x) = \exp\{\widehat{\eta}(x) + \widehat{\sigma} q_{\varepsilon,0.5}\}$, where $q_{\varepsilon,0.5}$ is the error median. The default risk score is $-\widehat{\eta}(x)$. Parametric curves can be evaluated beyond the largest observed follow-up, but this is model-based extrapolation, not additional information supplied by censored observations.

3.2. Penalized semiparametric accelerated failure time models

The penalized AFT implementation does not maximize a penalized version of Equation 8. It uses a rank-based Gehan objective, without specifying the error distribution. Let $e_i(\beta) = \log Y_i - x_i^\top \beta$ and $(a)_+ = \max(a, 0)$. A normalized Gehan loss is

$$L_G(\beta) = \frac{1}{n^2} \sum_{i=1}^{n} \sum_{j=1}^{n} \delta_i \left(e_j(\beta) - e_i(\beta)\right)_+ . \tag{9}$$

An intercept cancels from the residual differences and is not identified by this criterion. For $\lambda > 0$ and $0 \leq \alpha \leq 1$, define

$$P_{\lambda,\alpha}(\beta) = \lambda \left\{ \alpha \|\beta\|_1 + \frac{1-\alpha}{2} \|\beta\|_2^2 \right\}, \quad \widehat{\beta} = \arg\min_{\beta} \{L_G(\beta) + P_{\lambda,\alpha}(\beta)\}. \tag{10}$$

The choices $\alpha = 1$ and $\alpha = 0$ give LASSO and ridge penalties; intermediate values give the elastic net (Zou and Hastie 2005). LASSO can set coefficients to zero, whereas ridge shrinks them without imposing sparsity in the usual continuous setting.

The solver included in the package was adapted from **penAFT**. It uses the alternating direction method of multipliers to minimize this nonsmooth criterion (Suder and Molstad 2022; Molstad and Suder 2025). Predictors are standardized using the training sample, and the solver does not standardize them again. The three penalized AFT methods differ in α and return the score $-x^\top \widehat{\beta}$. They do not estimate an error distribution or its location, so survival probabilities and median times are unavailable. The scale of λ depends on the normalization of the loss and on the solver. Thus, the same value need not give the same amount of shrinkage in another model family.

3.3. Cox models and regularization

The proportional hazards formulation is

$$h(t \mid x) = h_0(t) \exp(x^\top \beta), \quad S(t \mid x) = \exp\{-H_0(t) \exp(x^\top \beta)\}. \tag{11}$$

For distinct failure times, the partial log-likelihood is given by

$$\ell_P(\beta) = \sum_{i:\delta_i=1} \left[x_i^\top \beta - \log \left\{ \sum_{j \in R(Y_i)} \exp(x_j^\top \beta) \right\} \right], \quad R(t) = \{j : Y_j \geq t\}. \tag{12}$$

The baseline hazard cancels from Equation 12; it need not be specified to estimate β. The ordinary Cox model uses `coxph()` with the Efron approximation for tied failures. A coefficient

gives a conditional hazard ratio $\exp(\beta_j)$ under the model, not a constant ratio of failure probabilities.

For penalized Cox estimation, the objective has the form

$$\widehat{\beta} = \arg\min_{\beta}\{-a_n \ell_P(\beta) + P_{\lambda,\alpha}(\beta)\}, \tag{13}$$

where $a_n > 0$ denotes the loss normalization used by the engine. The **glmnet** implementation uses coordinate-descent procedures and its own training standardization (Simon, Friedman, Hastie, and Tibshirani 2011; Friedman, Hastie, and Tibshirani 2010). The package requests Breslow ties for these fits. Consequently, an ordinary Cox fit and a weakly penalized fit can differ because of the tie convention as well as the penalty. The identifiers ending in `lasso`, `ridge`, and `elasticnet` correspond to $\alpha = 1$, 0, and an adjustable value, respectively.

For a fitted risk predictor $\widehat{\eta}(x)$, a Breslow baseline is

$$\widehat{H}_0(t) = \sum_{j:t_j \le t} \frac{d_j}{\sum_{i \in R(t_j)} \exp\{\widehat{\eta}(x_i)\}}, \quad \widehat{S}(t \mid x) = \exp\{-\widehat{H}_0(t)\exp\{\widehat{\eta}(x)\}\}. \tag{14}$$

For ordinary and penalized Cox models, baseline estimation and prediction use the corresponding engine routines. The score is $\widehat{\eta}(x)$. Proportional hazards remains an assumption after regularization; a penalty does not remove it.

3.4. Native neural Cox estimation

The neural Cox model replaces the linear predictor in Equation 11 by a neural network, while retaining proportional hazards (Katzman, Shaham, Cloninger, Bates, Jiang, and Kluger 2018). With one hidden layer of m units,

$$z_i = \tanh(W^\top \widetilde{x}_i + b), \qquad \eta_i = f_\theta(x_i) = v^\top z_i + c, \qquad h(t \mid x_i) = h_0(t)\exp(\eta_i), \tag{15}$$

where $\widetilde{x}_i$ uses training standardization and θ contains all weights and biases. Differences in network outputs are log hazard ratios; individual weights are not. The additive location is absorbed by the baseline.

Tied failures and the baseline

Let t_g denote a distinct observed failure time, D_g its failure set, $d_g = |D_g|$, and $R_g = \{i : Y_i \ge t_g\}$. Define $S_g = \sum_{i \in R_g} \exp(\eta_i)$ and $E_g = \sum_{i \in D_g} \exp(\eta_i)$. With $N_E = \sum_i \delta_i$, the penalized objective used by the native implementation is

$$\mathcal{L}(\theta) = \frac{1}{N_E}\left[\sum_g \sum_{l=0}^{d_g-1} \log\{S_g - (l/d_g)E_g\} - \sum_{i:\delta_i=1} \eta_i\right] + \frac{\lambda_N}{2}\|\theta\|^2. \tag{16}$$

Under Efron's approximation (Efron 1977), observations censored at a failure time remain in the risk set. Only contributions from failures are subtracted in the successive denominators. Breslow replaces the inner sum by $d_g \log S_g$; the expressions coincide without tied failures. Neither is the exact discrete partial likelihood, and times are not perturbed.

After estimating θ, we use the matching baseline increments

$$\Delta\widehat{H}_0(t_g) = \begin{cases} \displaystyle\sum_{l=0}^{d_g-1} \{\widehat{S}_g - (l/d_g)\widehat{E}_g\}^{-1}, & \text{Efron,} \\ d_g/\widehat{S}_g, & \text{Breslow.} \end{cases} \tag{17}$$

Summing these training-only increments gives $\widehat{S}(t \mid x) = \exp[-\widehat{H}_0(t)\exp\{f_{\widehat{\theta}}(x)\}]$. The calculations center the log risks and store the baseline on a logarithmic scale, with the corresponding adjustment at prediction. This avoids overflow without changing the curve.

Analytic derivatives and training

For Efron, write

$$A_g = \sum_{l=0}^{d_g-1} \frac{1}{S_g - (l/d_g)E_g}, \qquad B_g = \sum_{l=0}^{d_g-1} \frac{l/d_g}{S_g - (l/d_g)E_g}. \tag{18}$$

The derivative of the unpenalized part of Equation 16 with respect to a log risk is given by

$$\frac{\partial \mathcal{L}_{\text{data}}}{\partial \eta_i} = \frac{1}{N_E}\left[\exp(\eta_i)\sum_{g:i\in R_g} A_g - \delta_i - \delta_i \exp(\eta_i) B_{g(i)}\right], \tag{19}$$

where $g(i)$ is the failure group of an observed event; its term is zero for a censored row. Under Breslow, $A_g = d_g/S_g$ and $B_g = 0$. The observations are ordered and their time groups are defined once. Cumulative sums then give the risk-set contributions without an $n \times n$ array. Log-sum-exp calculations are used when risk values differ greatly. Efficient grouped Cox calculations with both Breslow and Efron ties have also been studied by Yang, Abraham, Kim, Smirnov, Ruan, Haibe-Kains, and Tibshirani (2022). The present contribution is their use in a R implementation with clearly defined predictions; efficient calculation with tied risk sets is not claimed as a new method.

Let $e_i = \partial \mathcal{L}_{\text{data}}/\partial \eta_i$ and $a_i = e_i v \odot (1 - z_i \odot z_i)$. Backpropagation gives

$$\begin{aligned} \nabla_W \mathcal{L} &= \sum_i \widetilde{x}_i a_i^\top + \lambda_N W, & \nabla_b \mathcal{L} &= \sum_i a_i + \lambda_N b, \\ \nabla_v \mathcal{L} &= \sum_i e_i z_i + \lambda_N v, & \partial_c \mathcal{L} &= \sum_i e_i + \lambda_N c. \end{aligned} \tag{20}$$

These expressions are evaluated using ordinary R matrix operations. The penalty includes biases as well as connection weights. Full-batch Adam uses the complete training gradient at every update (Kingma and Ba 2015):

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1-\beta_1) g_k, \qquad v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k^2, \\ \theta_k &= \theta_{k-1} - \rho \frac{m_k/(1-\beta_1^k)}{\sqrt{v_k/(1-\beta_2^k)} + \epsilon_A}. \end{aligned} \tag{21}$$

Squares, roots, and division in Equation 21 are componentwise. The implementation uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon_A = 10^{-8}$. Learning rate ρ, hidden size, penalty, and epoch

budget are explicit parameters. Fixed epochs do not certify convergence to a stationary point or a global optimum. The final loss, penalty, maximum absolute gradient, tie counts, and sample size remain available through `surv_diagnostics()`. No test-based early stopping is performed.

The fitted network is stored as ordinary R arrays and requires neither **torch**, **survdnn**, **reticulate**, nor Python. This belongs to the neural proportional hazards family used by DeepSurv, but does not reproduce its original architecture or tuning procedure. Section 8 separates numerical agreement from predictive performance.

3.5. Survival trees and random survival forests

A survival tree partitions the covariate space into disjoint terminal regions $A_1, \ldots, A_L$. Its prediction is constant within each terminal region. The `tree` adapter uses `rpart(method = "exp")`, an exponential-deviance construction (Breiman, Friedman, Olshen, and Stone 1984; Therneau and Atkinson 2026). It is not a log-rank tree. Let $\widetilde{H}_0(Y_i)$ denote the transformed exposure obtained from a training baseline. Within a node A, the unshrunk relative-rate estimate maximizes

$$\ell_A(\theta) = \sum_{i:x_i \in A} \{\delta_i \log \theta - \theta \widetilde{H}_0(Y_i)\}, \quad \widehat{\theta}_A = \frac{\sum_{i:x_i \in A} \delta_i}{\sum_{i:x_i \in A} \widetilde{H}_0(Y_i)}. \tag{22}$$

Terms not depending on θ are omitted. Equation 22 describes the unshrunk likelihood calculation. The engine additionally uses its default rate-shrinkage setting, `shrink = 1`, so returned leaf rates need not equal the unshrunk ratio. Splits improve the corresponding deviance, with complexity and node-size controls restricting the tree. If $D(\mathcal{T})$ is its deviance, cost-complexity pruning compares criteria of the form $D(\mathcal{T}) + \kappa|\mathcal{T}|$. The package returns the engine's risk score for each terminal node. It does not add a separate survival-curve estimator to this tree.

Random survival forests (RSF) combine trees fitted to bootstrap samples (Ishwaran, Kogalur, Blackstone, and Lauer 2008). Only a random subset of predictors is considered at each node. For a candidate split, a standardized log-rank statistic is

$$Z = \frac{\sum_j (d_{Lj} - n_{Lj} d_j / n_j)}{\left\{ \sum_{j:n_j > 1} \frac{n_{Lj} n_{Rj} d_j (n_j - d_j)}{n_j^2 (n_j - 1)} \right\}^{1/2}}. \tag{23}$$

Here n_{Lj} and n_{Rj} are the risk-set counts in the proposed children. The integrated RF-SRC core uses log-rank splitting; `mtry`, minimum node size, and candidate split points control the construction (Ishwaran and Kogalur 2026). For tree b and the terminal node containing x, a cumulative hazard is $\widehat{H}_b(t \mid x) = \sum_{t_j \le t} d_{bj}(x)/n_{bj}(x)$. Terminal estimates are averaged over trees:

$$\widehat{H}_{\mathrm{RSF}}(t \mid x) = B^{-1} \sum_b \widehat{H}_b(t \mid x), \quad \widehat{S}_{\mathrm{RSF}}(t \mid x) = B^{-1} \sum_b \widehat{S}_b(t \mid x). \tag{24}$$

In general, the average survival is not the exponential of the negative average cumulative hazard. The mortality score sums the ensemble cumulative hazard over the training event-time grid. This score orders units but is not a failure probability. Out-of-bag observations

can be used for internal forest evaluation. Here we select hyperparameters using the same validation folds as for the other methods.

3.6. Gradient boosting with a Cox objective

For tree learners g_m, a boosted predictor after M stages is

$$f_M(x) = f_0(x) + \nu \sum_{m=1}^{M} \rho_m g_m(x), \qquad \nu > 0. \tag{25}$$

Considering the negative partial log-likelihood $\mathcal{L}(f) = -\ell_P(f)$, a negative gradient at stage m is

$$u_i^{(m)} = \delta_i - \sum_{k:\delta_k=1,\, Y_k \le Y_i} \frac{\exp\{f_{m-1}(x_i)\}}{\sum_{j \in R(Y_k)} \exp\{f_{m-1}(x_j)\}}. \tag{26}$$

Equation 26 is written for distinct event times. A weak tree approximates the gradient and a loss-based step updates the predictor. The stages, depth, learning rate, and subsampling fraction jointly control the fit (Friedman 2001). The `boosting` identifier uses `gbm(distribution = "coxph")` and returns its log-risk predictor (Ridgeway and Developers 2026). This is gradient boosting with tree learners, not componentwise likelihood-based Cox boosting (Binder and Schumacher 2008). A nonlinear predictor in Equation 11 still implies proportional hazards without a time-dependent predictor. The package returns a risk score only for this method; it does not add a baseline hazard or a calibration model.

3.7. XGBoost survival objectives

The **xgboost** engine builds regularized additive trees using gradient and curvature information (Chen and Guestrin 2016; Chen, He, Benesty, Khotilovich, Tang, Cho, Chen, Mitchell, Cano, Zhou, Li, Xie, Lin, Geng, Li, Yuan, and Cortes 2026). For tree $g_m(x) = w_{m,q_m(x)}$, with leaf assignment q_m, a stagewise objective has the form

$$\mathcal{J}_m = \mathcal{L}(f_{m-1} + g_m) + \Omega(g_m), \quad \Omega(g_m) = \gamma L_m + \frac{\lambda_w}{2} \sum_{\ell=1}^{L_m} w_{m\ell}^2. \tag{27}$$

Second-order approximations are used to choose splits and estimate leaf values. The package uses histogram-based tree construction and allows the user to set the number and depth of trees, learning rate, minimum child weight, and leaf penalty. Not every XGBoost option is included.

For `xgboost_cox`, the Cox loss gives a log-risk predictor, called the margin in XGBoost. A failure is supplied as $+Y_i$ and a censored observation as $-Y_i$. The package estimates the Breslow baseline in Equation 14 using training outcomes only, combining tied event counts. For numerical evaluation, store the training center $c = \min_i \widehat{\eta}_i/2 + \max_i \widehat{\eta}_i/2$ and write $z_i = \widehat{\eta}_i - c$. The halved-endpoint calculation avoids overflowing their sum. Let $a_g = \mathrm{LSE}\{z_i : i \in R_g\}$, where $\mathrm{LSE}(u_1, \ldots, u_k) = m + \log \sum_j \exp(u_j - m)$ and $m = \max_j u_j$. We compute

$$\ell_g = \log d_g - a_g, \qquad L(t) = \mathrm{LSE}\{\ell_g : t_g \le t\}, \qquad \widehat{S}(t \mid x) = \exp[-\exp\{\widehat{\eta}(x) - c + L(t)\}]. \tag{28}$$

A time with no events has $\ell_g = -\infty$. Before the first event, $L(t) = -\infty$ and survival is one. Both risk-set sums and cumulative increments are evaluated on the logarithmic scale. Centering prevents a large common margin from hiding small log-count corrections, but cannot

recover information already lost in the supplied margins. Centering once and then exponentiating all values is not sufficient: the largest margin can leave a later risk set. Equation 28 therefore uses each risk set separately, preserving the Breslow estimator without clipping margins or changing the Cox objective.

For `xgboost_aft`, a nonlinear predictor replaces the linear location in Equation 7. The response is supplied through bounds:

$$[L_i, U_i] = \begin{cases} [Y_i, Y_i], & \delta_i = 1, \\ [Y_i, \infty), & \delta_i = 0. \end{cases} \tag{29}$$

The loss uses a density for exact events and a survival probability for censoring (Barnwal, Cho, and Hocking 2022). The error distribution is normal, logistic, or minimum extreme value; its scale is a tuning parameter, not an additional parameter estimated automatically by the wrapper. The initial log-location is $n^{-1}\sum_i \log Y_i$, computed in training so it shifts with the time unit. This initializes optimization; it is not an estimate of mean lifetime. For log-time location $f(x)$, the score is $-f(x)$ and

$$\widehat{S}(t \mid x) = S_\varepsilon\left(\frac{\log t - f(x)}{\sigma}\right), \quad \widehat{m}(x) = \exp\{f(x) + \sigma q_{\varepsilon,0.5}\}. \tag{30}$$

For normal and logistic errors, $q_{\varepsilon,0.5} = 0$. For minimum extreme-value errors, $q_{\varepsilon,0.5} = \log(\log 2)$. Exponentiating the margin alone does not give the median in the latter case.

3.8. Multi-task logistic regression

Multi-task logistic regression (MTLR) estimates a distribution over ordered failure-time intervals (Yu, Greiner, Lin, and Baracos 2011). Let $0 = \tau_0 < \cdots < \tau_K$ be a training-defined grid, with a final interval beyond τ_K. Define $a_k(x) = \sum_{j=k}^{K}(b_j + x^\top\theta_j)$ for $k = 1, \ldots, K+1$, with $a_{K+1}(x) = 0$. The interval probabilities are

$$p_k(x) = \frac{\exp\{a_k(x)\}}{\sum_{\ell=1}^{K+1}\exp\{a_\ell(x)\}}, \quad S(\tau_j \mid x) = \sum_{k=j+1}^{K+1} p_k(x). \tag{31}$$

The construction restricts possible survival-status sequences; it is not a collection of unrelated binary regressions. For an event in interval k_i, the contribution is $p_{k_i}(x_i)$; for censoring at boundary τ_{j_i}, it is $\sum_{k>j_i} p_k(x_i)$. A representative penalized criterion is

$$-\sum_i \left\{\delta_i \log p_{k_i}(x_i) + (1-\delta_i)\log\sum_{k>j_i} p_k(x_i)\right\} + \frac{\lambda}{2}\sum_{j=1}^{K}\|\theta_j\|_2^2. \tag{32}$$

The **MTLR** engine determines how observations between knots are encoded. The original formulation also considered smoothing adjacent parameters. Here we use the coefficient penalty `C1`, called `lambda` in **survlearning** (Haider 2026). Estimation uses L-BFGS-B, with an initial fitting stage before minimizing the censored criterion. Although the uncensored multinomial form is convex, this does not establish global convexity of the censored criterion.

The package returns survival probabilities interpolated by **MTLR**. For concordance, it uses $r_{\mathrm{MTLR}}(x) = -\widehat{\mu}_\tau(x)$, with $\tau = \max_{i\in\mathcal{D}_{\mathrm{train}}} Y_i$. The integral in Equation 5 is approximated by

the trapezoidal rule on 101 equally spaced points together with learned knots. The horizon and grid are recomputed within each training fold. This score does not impose proportional hazards or estimate a mean beyond the chosen horizon. When curves cross, the score need not order risks in the same way at every horizon. Full means and medians requiring tail extrapolation are not returned.

3.9. Support vector learning objectives

For SVMs, we separate the choice of kernel from the choice of loss function. Linear and additive kernels are available for four objectives: censored regression (`svm`), comparable-pair ranking (`svm_rank`), a ranking–regression hybrid (`svm_hybrid`), and censoring-weighted regression (`svm_ipcw`). These are distinct estimators, not alternative solvers for one loss. All return risk scores, with larger values indicating greater risk; they do not return survival curves. The comparison in Section 6.2.1 keeps the covariates, training rows, and validation folds fixed to distinguish these choices.

Censored absolute-loss regression

The `svm` identifier fits zero-epsilon censored support vector regression (Fouodo, König, Weihs, Ziegler, and Wright 2018). For $g(x) = b + \langle w, \phi(x)\rangle$, it minimizes

$$\frac{1}{2}\|w\|^2 + C\sum_i \{(Y_i - g(x_i))_+ + \delta_i\,(g(x_i) - Y_i)_+\}. \tag{33}$$

For an observed failure, the loss is the absolute prediction error. For a censored observation, only predictions below the observed time are penalized. Thus, this is not ordinary regression on recorded times, and the fitted value is not automatically a calibrated conditional median. With heavy administrative censoring, the loss may favor predictions close to the end of observation, even when those predictions give poor risk ordering.

The kernels available through the interface are

$$K_{\text{lin}}(x,z) = x^\top z, \qquad K_{\text{add}}(x,z) = \sum_{j=1}^{p}\left(1 - \frac{|x_j - z_j|}{R_j}\right), \quad R_j = \max_{\text{train}} x_j - \min_{\text{train}} x_j. \tag{34}$$

Predictor coding, removal of constant columns, and standardization use training observations only. The additive ranges are also computed in training and remain fixed during prediction, even when a predictor is constant among the observations with nonzero dual coefficients.

The dual of Equation 33 is

$$\min_c \ \tfrac{1}{2}c^\top K c - Y^\top c, \qquad \mathbf{1}^\top c = 0, \quad -C\delta_i \le c_i \le C. \tag{35}$$

For $c = \alpha - D\beta$, with $D = \text{diag}(\delta)$, its Hessian is $[I, -D]^\top K[I, -D]$. It is positive semidefinite by construction; no positive-definite projection changes the objective.

Both kernels admit an exact sparse primal. The linear case has p penalized coefficients. For the additive case, the identity $1 - |u - v| = 1 - u - v + 2\min(u, v)$, on normalized training coordinates, gives a piecewise-linear component at every distinct training value. The terms $1 - u - v$ vanish in the dual quadratic form because $\mathbf{1}^\top c = 0$. After optimizing over the

unpenalized constant, the equivalent anchored representation has penalty

$$\frac{1}{4}\sum_{j}\sum_{k=1}^{m_j}\frac{(\theta_{jk}-\theta_{j,k-1})^2}{u_{jk}-u_{j,k-1}}, \qquad \theta_{j0}=0. \tag{36}$$

The design matrix is sparse, and each penalty block is tridiagonal. Every observation retains its own loss constraints, and every distinct training coordinate is included. Thus, neither the kernel nor the sample is reduced to obtain this representation. Clarabel is the default solver and OSQP an explicit alternative (Goulart and Chen 2026). Appendix A derives the representation, its prediction rule and independent numerical certificate.

The returned risk score is $-\widehat{g}(x)$, not a survival probability or a calibrated lifetime. Ranking SVMs minimize a different criterion (Pölsterl, Navab, and Katouzian 2015) and do not replace Equation 33. We select cost separately for each kernel using training-fold validation. When predictions are nearly tied, convergence of the objective alone does not ensure stable concordance. The default prediction resolution is $10^{-8}\max_i Y_i$, fixed in training; setting `prediction_tolerance = 0` exposes raw scores without changing fitting. This convention and its sensitivity must accompany comparisons under heavy censoring.

Heavy censoring also affects what this loss estimates, even when the optimization is exact. To see this, suppose that censoring occurs at a fixed horizon τ and that the conditional failure distribution is continuous. For $a<\tau$, the derivative of the unpenalized conditional expected loss is $2F_T(a \mid x)-1$; for $a>\tau$, it is $F_T(\tau \mid x)$. Consequently, if $0<F_T(\tau \mid x)<1/2$, the loss is minimized at $a=\tau$, not at the unobserved conditional median. Groups with different failure distributions can therefore have the same optimal regression score. This does not describe every fitted kernel model, but explains why accurate optimization and a larger sample need not give high concordance. Oversampling failures changes the distribution of the empirical loss. It is therefore not equivalent to fitting the full cohort and must be reported when comparing implementations or examples.

Exact comparable-pair ranking

Let $\mathcal{P}=\{(i,j):\delta_i=1, Y_i<Y_j\}$ and let $r_i=r(x_i)$. The ranking estimator minimizes

$$Q_\lambda(r)=\frac{\lambda}{2}\|r\|_{\mathcal{H}}^2+L_{\text{rank}}(r), \qquad L_{\text{rank}}(r)=\frac{1}{|\mathcal{P}|}\sum_{(i,j)\in\mathcal{P}}(1-r_i+r_j)_+^2. \tag{37}$$

The loss favors a larger risk score for the unit that fails first. A censored observation can be the longer-followed member of a pair. Tied observed times are excluded rather than perturbed. Since the intercept cancels from each margin, it is omitted. The squared-hinge loss approximates the ordering objective; it does not directly maximize a differentiable C-index. In particular, the training set $\mathcal{P}$ excludes equal-time event/censoring pairs admitted by the evaluation set $\mathcal{P}_H$.

Efficient primal survival SVM algorithms have been studied by Pölsterl *et al.* (2015). Here two time-ordered Fenwick sweeps evaluate the exact loss and its score gradient in $O(n\log n)$ time per evaluation. The score moments are calculated after removing a common offset, which cancels from every margin and avoids subtracting large nearly equal terms. For example, an active pair contributes $-2\,(1-r_i+r_j)_+/|\mathcal{P}|$ to the derivative with respect to r_i, and the opposite contribution to that for r_j. All comparable pairs are retained without constructing a

pair-design matrix. For the additive kernel, the RKHS norm is understood modulo a constant, as in the preceding anchored representation. Pair losses are invariant to that constant, so fixing the anchor does not restrict risk orderings.

For the additive representation in Equation 36, write

$$\theta_{jk} = \sum_{\ell \le k} \sqrt{2(u_{j\ell} - u_{j,\ell-1})}\, a_{j\ell}. \tag{38}$$

The RKHS norm becomes $\|a\|^2$, so that L-BFGS-B uses an analytic gradient in Euclidean coordinates. Every distinct training knot remains. For pure ranking, strong convexity also gives

$$Q_\lambda(a) - Q_\lambda(a^*) \le \frac{\|\nabla Q_\lambda(a)\|^2}{2\lambda}. \tag{39}$$

Let ε_{opt} denote `tolerance` and v the fitted parameter vector. Acceptance requires finite parameters and objective, and $\|\nabla Q(v)\|_2 \le \varepsilon_{\text{opt}} \max(1, \|v\|_2)$. Pure ranking additionally requires the bound in Equation 39 to be at most $\varepsilon_{\text{opt}} \max(1, |Q_\lambda(v)|)$. A line-search termination at floating-point precision is accepted only when these same accuracy checks pass; it is labeled `stationarity_verified`, with the original optimizer code and message retained. Iteration-limit and other optimizer failures remain ineligible. Linear interpolation and constant extension outside training ranges are preserved. The additive kernel supplies nonlinear main effects, not interactions.

A squared-loss hybrid

Let $z_i = Y_i / \max_{j \in \text{train}} Y_j$ and $r_i = g(x_i) + b$. The hybrid objective is

$$\frac{\lambda}{2}\|g\|_{\mathcal{H}}^2 + \eta L_{\text{rank}}(r) + \frac{1-\eta}{n} \sum_i \left\{ \delta_i (r_i + z_i)^2 + (1 - \delta_i)\, (r_i + z_i)_+^2 \right\}, \quad 0 < \eta < 1. \tag{40}$$

The parameter `rank_weight` controls η. The time scale is computed within each training fold, and the intercept is not penalized. The criterion combines ranking with a squared one-sided response loss, so it differs from the absolute-loss estimator. Ranking–regression combinations are established methods (Van Belle, Pelckmans, Van Huffel, and Suykens 2011; Fouodo *et al.* 2018). The unpenalized intercept means that Equation 39 is not asserted for the complete hybrid parameter vector.

IPCW learning for a restricted response

For a horizon τ specified in observed-time units, define $Z_i = \min(Y_i, \tau)/\tau$ and $A_i = I(Y_i \ge \tau) + \delta_i I(Y_i < \tau)$. The restricted response is observed when $A_i = 1$. With $G(t^- | x) = P(C \ge t \mid X = x)$, the weighted objective is

$$\frac{\lambda}{2}\|g\|_{\mathcal{H}}^2 + \frac{1}{n} \sum_i \frac{A_i}{\widehat{G}(\min(Y_i, \tau)^- | x_i)} \left(|Z_i + r_i| - \epsilon\right)_+^2, \qquad r_i = g(x_i) + b. \tag{41}$$

Conditionally independent censoring and positivity justify recovery of the corresponding complete-data loss by inverse-probability weighting (Goldberg and Kosorok 2017). At $\epsilon = 0$, the criterion is weighted kernel ridge regression for a restricted mean response. With positive epsilon it has a squared epsilon-insensitive target, not exactly that conditional mean. The

conditional mean is the unrestricted, unpenalized population target; regularization and the fitted function class need not recover it exactly. The package exposes a risk score rather than a full survival distribution.

The censoring model used to compute the weights is refitted in every training fold. Setting `censoring_method` to `"km"` gives marginal reverse Kaplan–Meier weights; `"cox"` gives conditional Cox weights. Marginal weights require appropriate marginal censoring assumptions; conditional weights also require a suitable conditional model. Observations censored early remain in the censoring fit but receive zero weight in the response loss. They are not removed to reduce computation. A censoring probability below `min_g`, default 0.05, gives an error rather than clipped weights. A smooth fitted censoring curve cannot supply follow-up that the study design does not allow. The empirical check covers the horizon for every training row and each observed restricted-response time, not only rows with positive loss weight. For all three alternatives, the default gradient tolerance is 10^{-6} and the prediction grid has dimensionless spacing 10^{-8}. This differs from the time-scaled grid of the original absolute-loss regression. Failed numerical checks make a tuning candidate ineligible. No additional package, Python installation or Torch runtime is required. Fixed-setting examples are given below; parameter selection uses training folds rather than these illustrative choices.

```
R> rank_fit <- surv_fit(form, train, "svm_rank",
+    params = list(kernel = "additive", lambda = 0.001))
R> hybrid_fit <- surv_fit(form, train, "svm_hybrid",
+    params = list(kernel = "additive", lambda = 0.001,
+                  rank_weight = 0.75))
R> ipcw_fit <- surv_fit(form, train, "svm_ipcw",
+    params = list(kernel = "linear", lambda = 0.1,
+                  horizon = 90, epsilon = 0))
R> surv_predict(hybrid_fit, test, type = "risk")
```

3.10. Other survival models

For `cox_spline`, the Cox linear predictor is replaced by $B_d(x)^\top \beta$, where natural-spline bases are learned from training numeric predictors, with factors retained. Estimation still uses the Cox partial likelihood. The `flexsurv` identifier uses an integrated flexible parametric spline likelihood; on the hazard scale used here, $\log H(t \mid x) = s(\log t; \gamma) + x^\top \beta$. Likelihood contributions follow Equation 3 (Jackson 2016). Training standardization and analytic gradients support optimization; baseline monotonicity is checked between knots and in the tails. Spline flexibility does not remove proportional hazards on this scale.

The `coxboost` adapter uses componentwise likelihood-based boosting (Binder and Schumacher 2008), rather than the regression-tree gradient boosting described above. At each step, a coordinate update is selected by its penalized partial-likelihood improvement and added to the current predictor; the number of steps controls regularization. The `ranger` and `aorsf` adapters provide alternative survival forests. The former uses axis-aligned splits; the latter searches oblique splits of the form $x^\top \beta \leq c$ (Wright and Ziegler 2017; Jaeger, Welden, Lenoir, and Pajewski 2022; Jaeger, Welden, Lenoir, Speiser, Segar, Pandey, and Pajewski 2023). For both forests, predictions use the engine's ensemble hazard and survival estimates rather than a replacement Cox baseline.

Finally, `neural_discrete` uses a single-hidden-layer **nnet** model for person-period Bernoulli responses, following the partial logistic approach (Biganzoli, Boracchi, Mariani, and Marubini 1998; Ripley and Venables 2002). If $h_k(x)$ is the conditional probability of an event in interval k, then

$$S(\tau_j \mid x) = \prod_{k \le j} \{1 - h_k(x)\}, \quad \ell = \sum_{(i,k) \in \mathcal{O}} \{d_{ik} \log h_k(x_i) + (1 - d_{ik}) \log[1 - h_k(x_i)]\}. \tag{42}$$

The set $\mathcal{O}$ contains periods with observed event/survival status. For a censored unit, only completed intervals contribute; an incomplete censoring interval is not labeled as an observed nonevent. Grouping time loses information within an incomplete interval, so this is not the continuous-time likelihood. The grid is defined from training data, and prediction returns stepwise curves on that grid. This differs from the native continuous-time neural Cox model in Section 3.4.

3.11. Convex survival stacking

Models with native survival curves can be combined through `surv_stack()`, following Wey, Connett, and Rudser (2015). Let $\widehat{S}_m^{(-k(i))}(t \mid x_i)$ be the out-of-fold (OOF) prediction from method m, fitted without the fold containing unit i. At fixed horizons $0 < t_1 < \cdots < t_J$, the weights solve

$$\widehat{w} = \underset{w_m \ge 0, \sum_m w_m = 1}{\arg\min} \frac{1}{nJ} \sum_{i=1}^{n} \sum_{j=1}^{J} \omega_i^{(-k(i))}(t_j) \left\{ I(Y_i > t_j) - \sum_m w_m \widehat{S}_m^{(-k(i))}(t_j \mid x_i) \right\}^2. \tag{43}$$

Here $\omega_i(t) = \delta_i I(Y_i \le t)/\widehat{G}(Y_i- \mid x_i) + I(Y_i > t)/\widehat{G}(t \mid x_i)$. Censoring estimation and preprocessing use each training fold only. Here G is the censoring survival function, under the assumptions in Section 4.5. Supplied base-model grids are selected by inner Harrell-C validation; otherwise their settings are fixed. All observations remain in the calculation, including censored units with zero loss weight at a given horizon. Unsupported horizons or failed fits stop estimation. Clarabel solves the convex quadratic problem, and the simplex gradient gap $w^\top \nabla Q(w) - \min_m \nabla_m Q(w)$ checks accuracy.

The base models are then refitted on all training observations, giving $\widehat{S}_{\text{stack}}(t \mid x) = \sum_m \widehat{w}_m \widehat{S}_m(t \mid x)$. Weights remain fixed over time, preserving probability bounds and nonincreasing curves. They need not be unique for duplicate predictions. Risk is negative restricted mean, computed by trapezoidal integration over zero plus the declared horizons. Risk-only SVMs are not assigned probability curves. The weight-training loss is not independent assessment of the stack; the complete procedure requires a separate test sample or outer validation.

4. Software design and use

4.1. Installation, method discovery, and fitting

The package requires R 4.3.0 or later. Modeling engines and supported solvers are installed as required dependencies. Development tools and **survivalsvm**, used for independent comparisons, are suggested packages. The following installation includes the engines needed to fit all methods:

```
R> install.packages("remotes")
R> remotes::install_local("survlearning_0.3.5.tar.gz",
+    dependencies = NA, upgrade = "never")
R> library("survlearning")
R> surv_methods()
```

The catalogue reports engines, availability and prediction types. `install_local()` resolves dependencies, unlike base installation with `repos = NULL`. Compiled dependencies can require a toolchain. Fitting never installs software or downloads data; Torch and Python are not required.

The common fitting signature is

```
surv_fit(formula, data, method = "cox", params = list(),
         seed = NULL, impute = "none")
```

The response is `Surv(time, event)`, with positive times and binary event indicators. Predictors can include factors, pointwise transformations, and interactions. Missing values give an error unless `impute = "median"` is requested. In that case, numeric medians and factor modes are estimated from training data; outcomes are never imputed. New factor levels and unsupported formula terms also give errors. In a dot formula, `.` is expanded using the original response, before any reversal of the censoring indicator. This prevents time and status from entering as predictors. Counting-process responses, offsets, strata, clusters, frailties, and time-dependent effects are not supported. Objects retain the design, settings, engine, counts, follow-up, seed and fitting time. Direct engine calls bypass the package's prediction conventions.

4.2. Preprocessing and parameter translation

Formula terms, factor levels, contrasts, nonconstant columns, and scaling values are stored during fitting and reused for new observations. SVM, Gehan AFT, MTLR, and neural Cox use training means and standard deviations; **glmnet** standardizes internally. Imputation and predictor coding are repeated within each training fold. These steps do not solve informative missingness or remove future information already used to construct a predictor.

Common names such as `trees`, `depth` and `learning_rate` retain objective-specific meanings and penalty scales. Unknown names give errors; defaults and engine seeds are recorded. With an explicit seed, **withr** restores the caller's random-number state even after an error (Hester, Henry, Muller, Ushey, Wickham, and Chang 2026); otherwise the current state is used. Reuse requires an explicit seed. A common seeded plan gives matched folds. Temporary engine options are also restored.

4.3. A common tuning procedure

The `surv_tune()` function adds a parameter `grid`, the number of `folds`, and a selection `metric` to the fitting call. A named list generates all combinations, whereas a data frame specifies particular combinations. With `grid = NULL`, only one fixed setting is evaluated. The same seed and outcome order give the same event-stratified folds, and every candidate is evaluated on all folds.

Let $\mathcal{D}_1, \ldots, \mathcal{D}_K$ denote held-out folds and ϑ a candidate setting. The selection rule is

$$\overline{C}(\vartheta) = \frac{1}{K} \sum_{k=1}^{K} \widehat{C}\{\mathcal{D}_k, r_{\widehat{f}_{\vartheta,-k}}\}, \qquad \widehat{\vartheta} = \underset{\vartheta \in \mathcal{G}_{\text{eligible}}}{\arg\max}\, \overline{C}(\vartheta). \tag{44}$$

Predictor coding and standardization use only $\mathcal{D} \setminus \mathcal{D}_k$. A setting can be selected only if every fold gives an accepted fit and finite concordance. Errors are retained in `fold_results`, rather than replaced by zero or omitted when computing the mean. If eligible means are tied, the first grid row is selected. The selected model is then refitted using all data supplied to `surv_tune()`, without using the external test sample.

The object retains `best_model`, `best_params`, candidate and fold results, the grid, and fold identities. Selection concordance is not an unbiased assessment of the search; an independent test sample or nested resampling is needed. `surv_resamples()` and `surv_validate()` also support grouped and forward-temporal evaluation of prespecified models. Nested tuning is available for stratified outer plans only, not grouped or temporal plans.

In `surv_validate()`, selection `metric` is independent of reporting `metrics`: error criteria are minimized and discrimination criteria maximized. AUC or Brier selection requires one `selection_time` when several horizons are reported. In `surv_tune()`, `metric` is scalar and AUC/Brier selection uses one `times` value. Reordering reported metrics cannot change the selection rule.

For a training formula `form` and data `train`, native neural Cox and optimized additive SVM fits use the same calling pattern:

```
R> nn <- surv_tune(form, train, method = "neural_cox",
+    grid = list(hidden = c(8L, 16L)), folds = 3, seed = 20261023,
+    params = list(epochs = 100L, ties = "efron",
+                  learning_rate = 0.001, decay = 0.0001))
R> sv <- surv_tune(form, train, method = "svm",
+    grid = list(cost = c(0.01, 0.1)), folds = 3, seed = 20261023,
+    params = list(kernel = "additive", solver = "clarabel"))
R> surv_diagnostics(sv)
```

These diagnostics describe the numerical fit, not its predictive quality. The package does not switch solvers automatically or treat a completed neural epoch budget as proof of convergence.

4.4. Prediction and scoring

`summary()` reports counts, parameters, capabilities, numerical status and time; tuned objects also report candidate results and failures. `coef()` and `vcov()` support ordinary Cox and parametric AFT, with log-hazard and log-time interpretations. Their Wald intervals exclude selection uncertainty. Other learners are not assigned coefficient tests. Section 8 checks agreement with direct engine calls.

For a fitted or tuned object, the common prediction signature is

```
surv_predict(object, newdata, type = "risk", times = NULL)
```

The `predict()` generic gives the same result. Risk and median predictions are vectors. Survival predictions are matrices with one row per unit and one column per requested horizon,

preserving order and repeated times. Prediction needs only covariates, whereas `surv_score()` also needs outcomes. Unsupported prediction types give an error. For semiparametric and nonparametric curves, horizons beyond training follow-up return `NA` with a warning. Parametric AFT predictions can extrapolate under the assumed tail model. Scores computed on training data are not out-of-sample assessments.

4.5. Evaluation with conditional censoring weights

Several evaluation criteria require an estimate of the censoring distribution, even when prediction from the event model does not. Define $G(t \mid x) = \Pr(C > t \mid x)$ and $G(t- \mid x) = \Pr(C \geq t \mid x)$. Under conditional independent censoring, positive censoring probabilities, and a reference distribution that applies to the assessment sample, inverse-probability weights recover contributions missing through censoring (Gerds and Schumacher 2006; Gerds, Kattan, Schumacher, and Yu 2013). A marginal censoring estimator generally does not replace $G(t \mid x)$ when censoring depends on the predictors.

The `surv_censor_fit()` interface estimates a marginal reverse KM, joint-stratum reverse KM, or a Cox working model for censoring. Its formula contains the original failure response `Surv(time, event)`; the caller does not reverse the indicator. For joint categorical strata, separate KM curves are estimated without imposing proportional hazards. For the Cox option, $h_C(t \mid x) = h_{C0}(t) \exp\{z(x)^\top \gamma\}$, with preprocessing learned from the reference data. Conditional independence, correct specification of this working model, and adequate stratum support remain assumptions.

Ties require care when times are recorded in days. At a time shared by failures and censorings, failures leave the risk set before the censoring hazard is evaluated. Consequently, a reverse-KM jump has denominator $n_j - d_j$, not n_j. The Cox implementation represents the same ordering by two ordered phases at each distinct recorded time: original failures depart at the first phase, and censoring events occur at the second. Efron's method handles tied censorings within that second phase. Baseline jumps are mapped back to the original time scale; the event model is not altered and no random time jitter is introduced.

For a native survival prediction $s_i(t) = \widehat{S}(t \mid x_i)$, the conditional-IPCW Brier estimator is

$$\widehat{\mathrm{BS}}(t) = \frac{1}{n} \sum_i \left\{ \frac{\delta_i \mathbb{I}(Y_i \leq t) s_i(t)^2}{\widehat{G}(Y_i- \mid x_i)} + \frac{\mathbb{I}(Y_i > t)\{1 - s_i(t)\}^2}{\widehat{G}(t \mid x_i)} \right\}. \tag{45}$$

Censorings at or before t contribute zero but remain in the denominator n. The integrated Brier score uses the normalized trapezoidal integral over the explicitly supplied horizon grid, not an implicit entire lifetime. Let $a_{ij} = \mathbb{I}(r_i > r_j) + \frac{1}{2}\mathbb{I}(r_i = r_j)$ and $w_i^E(t) = \delta_i \mathbb{I}(Y_i \leq t)/\widehat{G}(Y_i- \mid x_i)$, $w_j^D(t) = \mathbb{I}(Y_j > t)/\widehat{G}(t \mid x_j)$. The cumulative/dynamic AUC is

$$\widehat{\mathrm{AUC}}(t) = \frac{\sum_{i,j} w_i^E(t) w_j^D(t) a_{ij}}{\sum_{i,j} w_i^E(t) w_j^D(t)}. \tag{46}$$

Both cases and controls receive conditional weights. Sorted control ranks avoid constructing a dense pair matrix.

For concordance truncated at a prespecified τ, the separately named `ipcw_c` criterion uses

$$\begin{aligned} v_{ij}(\tau) &= \frac{\delta_i \mathbb{I}(Y_i \le \tau, Y_i < Y_j)}{\widehat{G}(Y_i- \mid x_i)\widehat{G}(Y_i \mid x_j)}, \\ \widehat{C}_{\mathrm{IPCW}}(\tau) &= \frac{\sum_{i,j} v_{ij}(\tau) a_{ij}}{\sum_{i,j} v_{ij}(\tau)}. \end{aligned} \tag{47}$$

The two factors correspond to observing the earlier failure and retaining the later unit beyond its time. Equal outcome times are excluded and risk ties receive half credit. This criterion is related to conditional-censoring concordance estimation (Gerds *et al.* 2013); its precise discrete tie convention is specified by Equation 47. Historical `uno` retains its marginal weight $1/\widehat{G}(Y_i-)^2$ and is rejected with conditional G. Even with marginal weights these formulas differ at censoring jumps; the names do not conceal that difference. Event-time-group processing avoids quadratic storage but can be slower than the marginal Uno rank-tree routine.

For the generated conditional-censoring example in Section 7.3, the reference fit is supplied explicitly:

```
R> ct_g <- surv_censor_fit(ct_formula, ct_train, method = "cox")
R> surv_evaluate(ct_fit, ct_test,
+    metrics = c("ipcw_c", "auc", "brier"),
+    times = 10, tau = 15, censoring_model = ct_g)
```

The assessment data do not replace the training reference. Unsupported horizons, unknown strata, or censoring probabilities below `min_g` give an error; weights are not clipped. Bootstrap samples resample assessment cases while keeping the event and censoring fits fixed, so their fitting uncertainty is excluded. During tuning and nested validation, the `censoring` argument instead receives a formula or list. The censoring model is then fitted separately in each training fold, including inner folds.

Grouped calibration uses the same explicit censoring fit when supplied. For a fixed prediction group B, the normalized IPCW estimate is

$$\widehat{S}_B(t) = \frac{\sum_{i \in B} w_i^D(t)}{\sum_{i \in B}\{w_i^E(t) + w_i^D(t)\}}. \tag{48}$$

The result is compared with the group's mean predicted survival, not with a risk score. This Hájek-type ratio differs from Kaplan–Meier and requires the censoring assumptions above and a positive denominator. Groups are formed separately at each horizon using midranks, so tied predictions stay together. The effective sample size is $(\sum b_i)^2 / \sum b_i^2$, with $b_i = w_i^E + w_i^D$. Sparse events, small risk counts and collapsed groups are reported. An optional within-group case bootstrap holds both fitted models and the original groups fixed and gives percentile intervals; undefined repetitions are counted. Without an explicit censoring fit, `surv_calibrate()` retains grouped KM with Greenwood intervals. The `conf_level` argument sets the confidence level in both branches. The IPCW output additionally reports `effective_n` and `bootstrap_valid`; no bootstrap intervals are reported when `n_boot = 0`.

```
R> surv_calibrate(ct_fit, ct_test, times = 10, groups = 4,
+    censoring_model = ct_g, n_boot = 200, seed = 14)
```

Neither estimator recalibrates a model. Its intervals do not include retraining, model selection, censoring-model estimation or full-pipeline uncertainty.

4.6. Paired intervals and calibration measures

For independent assessment units, `surv_compare()` can return $\widehat{\Delta} = \widehat{M}_A - \widehat{M}_B$ relative to a named reference. Each bootstrap sample uses the same unit indices for both models. The interval is given by the empirical quantiles $[q_{\alpha/2}(\widehat{\Delta}^*), q_{1-\alpha/2}(\widehat{\Delta}^*)]$. Event and censoring fits remain fixed. These are pointwise intervals, excluding refitting and selection uncertainty. Undefined repetitions are counted. Repeated CV rows must not be treated as independent test units, and no fold-level t test is used.

With `type = "metrics"`, `surv_calibrate()` evaluates the failure probabilities $p_i(t) = 1 - \widehat{S}(t \mid x_i)$. Weighted logistic calibration uses $z_i = I(Y_i \le t, \delta_i = 1)$, the IPCW weights above, and $\eta_i = \operatorname{logit} p_i(t)$:

$$(\widehat{a}, \widehat{b}) = \arg\max_{a,b} \sum_i \omega_i(t) \left[z_i \log \pi_i + (1 - z_i) \log(1 - \pi_i) \right], \qquad \operatorname{logit} \pi_i = a + b\eta_i. \tag{49}$$

The reported slope is $\widehat{b}$; the reported intercept comes from a separate fit with $b = 1$. Their ideal values are one and zero, respectively. A positive intercept indicates underprediction of failure; a slope below one suggests predictions that vary too much on the logit scale. The default censoring reference uses training outcomes; conditional models can be supplied. Bootstrap intervals resample assessment units with both models fixed. Exact boundary probabilities, separation and constant logits are diagnosed, not clipped. These measures evaluate rather than recalibrate the prediction model.

4.7. Interpreting a fixed fitted predictor

Permutation importance and first-order accumulated local effects (ALE) describe a fitted model without refitting it. The package identifies the original predictors from the training formula, including transformations and interactions, rather than treating encoded factor contrasts as separate variables. For a predictor group A, let $\mathcal{D}^{\pi_b(A)}$ permute its original columns jointly across assessment rows. The implemented importance is

$$\widehat{I}_A = \frac{1}{B} \sum_{b=1}^{B} \left[\widehat{C}\{\mathcal{D}, r\} - \widehat{C}\{\mathcal{D}^{\pi_b(A)}, r\} \right]. \tag{50}$$

The same permutations are used across predictor groups. Factor contrasts are not permuted separately, and formula transformations are recomputed after permuting the original columns. Importance can be negative. Its standard deviation describes variation across permutations for this fitted model and sample; it is not a confidence interval and does not include refitting uncertainty. Correlation between predictors complicates the interpretation, and the final test sample should not be reused to select variables.

ALE summarizes local prediction changes within observed covariate intervals (Apley and Zhu 2020). For numeric predictor q, let $z_0 < \cdots < z_K$ be empirical quantile boundaries and I_k its

bin membership. For target $f(x) = r(x)$ or $f(x) = 1 - \widehat{S}(t_0 \mid x)$, define

$$\begin{aligned} \widehat{d}_k &= \frac{1}{n_k} \sum_{i \in I_k} \{f(z_k, x_{i,-q}) - f(z_{k-1}, x_{i,-q})\}, \\ \widehat{A}_k &= \sum_{\ell=1}^{k} \widehat{d}_\ell - \frac{1}{n} \sum_{j=1}^{K} n_j \sum_{\ell=1}^{j} \widehat{d}_\ell. \end{aligned} \tag{51}$$

The values correspond to upper bin boundaries and are centered using the observed bin frequencies. Repeated boundaries and empty bins are merged. This first-order approximation does not add smoothing or confidence intervals and has no causal interpretation. ALE for failure probabilities requires a model that returns survival curves, whereas risk ALE uses the model's own score scale. The functions are called as follows:

```
R> importance <- surv_importance(forest, test,
+    n_repeats = 5, seed = 20261024)
R> effects <- surv_ale(forest, test,
+    variable = "temperature", bins = 10,
+    type = "failure", time = 180)
```

Base-graphics methods are supplied for both returned objects.

4.8. Reliability summaries at an attained time

For fixed covariates and attained model time a with $S(a \mid x) > 0$, the conditional survival of the remaining duration is

$$S_a(u \mid x) = \Pr(T - a > u \mid T > a, x) = \frac{S(a + u \mid x)}{S(a \mid x)}, \qquad u \geq 0. \tag{52}$$

The interface returns $F_a(u \mid x) = 1 - S_a(u \mid x)$ and

$$\mu_{a,\tau}(x) = \int_0^\tau S_a(u \mid x)\, du, \qquad q_{a,p}(x) = \inf\{u \in [0, \tau] : S_a(u \mid x) \leq 1 - p\}. \tag{53}$$

The restricted mean is computed by trapezoidal integration on a grid containing zero, the requested horizons, and equally spaced intermediate points. A quantile is the first grid point where the curve crosses the required level, not an exact failure time. If there is no crossing, the function returns `NA` and records the reason. Neither a restricted mean nor an unreached median estimates unrestricted life expectancy or cure.

The `age` argument refers to the model's time origin; for the present cohort this is time since first recorded observation, not installation age. Equation 52 conditions on survival while keeping covariates fixed. It is not a longitudinal updating model. By default, $a+\tau$ cannot exceed training follow-up. Only supported parametric engines allow an explicit `extrapolate = TRUE`; this adds a tail assumption, not observed information. Risk-only methods are rejected. For example,

```
R> remaining <- surv_reliability(forest, new_drives,
+    age = 30, horizons = c(60, 150), probs = c(0.01, 0.5),
+    grid_points = 501)
R> remaining$summary
R> remaining$quantiles
```

4.9. Local parallel tuning and checkpoint recovery

By default, tuning runs sequentially and writes no files. With `workers > 1`, local PSOCK processes from **parallel** fit separate folds and reuse compatible SVM matrices within each fold. Changing the execution order does not change seeds, candidate ordering, or the caller's random-number state. Explicit engine thread counts must be one. Starting workers, transferring objects, and duplicating memory can make parallel execution slower for small searches.

A `checkpoint` saves candidate-fold results and the final selected fit. Before reusing them, the package checks data and row order, formulas, folds, parameters, seeds, random-number and contrast settings, and software versions. Failed folds are not reused as successful fits and are retried only on request. Formulas passed to workers cannot contain arbitrary local functions. Missing fits rebuild their matrices rather than restore optimizer warm starts. Only one search can own a checkpoint directory. Errors and interrupts release its lock; after a hard crash, the user must check that the process has stopped before removing the lock. Corrupt or incompatible records give an error. This permits local recovery, not distributed fault tolerance.

```
R> tuned <- surv_tune(form, train, "svm",
+    grid = list(cost = c(0.01, 0.1)), folds = 3, seed = 20261023,
+    params = list(kernel = "linear", solver = "clarabel"),
+    workers = 2, checkpoint = "owned-search", progress = TRUE)
R> tuned$execution
```

Repeating the call reuses compatible completed folds and the selected fit. The output reports how many folds were computed and how many were reused.

5. A controlled example with covariate-based methods

5.1. Data generation and matched fitting

We consider one generated reliability data set to illustrate the available methods, rather than a repeated simulation study. There are 1,560 independent units with independent covariates: temperature $X_1 \sim N(30, 5^2)$, power-on hours $X_2 \sim U(1000, 30000)$, load $X_3 \sim U(0, 1)$, and binary design X_4 with equal probabilities. Let $Z_1 = (X_1 - 30)/5$ and $Z_2 = (X_2 - 15500)/14500$. Failure and censoring times are generated by

$$\log T = \log 90 - 0.5Z_1 - 0.25Z_2 - 0.45X_3^2 + 0.3Z_1X_3 + 0.25\mathbb{I}(X_4 = B) + 0.7\varepsilon, \quad \varepsilon \sim N(0, 1), \tag{54}$$

$$C = \min(C^*, 365), \qquad C^* \sim \text{Exp}(1/160). \tag{55}$$

The censoring variable is independent of covariates and failure time. The first 360 units form training and the remaining 1,200 form test. With seed 20261020, the samples contain 211 and 650 events, respectively. Training censoring is 41.4%. All 25 methods use the same training units, the same four covariates, and three-fold tuning with seed 20261021. The fitted formula does not explicitly include the squared load or interaction in Equation 54.

Small grids keep replication accessible. Gehan penalties use $\lambda \in \{0.01, 0.1\}$, Cox penalties use $\lambda \in \{0.001, 0.01\}$, and elastic-net candidates use $\alpha \in \{0.25, 0.75\}$. The parametric AFT candidates are Weibull, log-normal, and log-logistic. The tree uses complexity parameters 0.001, 0.01, or 0.05. The forest uses 200 trees and node sizes 15 or 50. Boosting considers 100 or 200 stages; XGBoost Cox also considers depths one or two, while XGBoost AFT considers normal-error scales 0.5 or 1. MTLR uses five or ten intervals and $\lambda \in \{0.1, 1\}$. The absolute-loss SVM grid evaluates the seven costs $10^{-3}, \ldots, 10^{3}$ for each linear and additive kernel. Both use the exact primal with Clarabel. The spline Cox degrees of freedom are two or three, flexible parametric interior-knot counts are zero or one, and componentwise CoxBoost uses 50 or 100 steps. The alternative forests use 200 trees, with node sizes 15 or 50 for **ranger** and 5 or 15 for **aorsf**. Native neural Cox considers 8 or 16 hidden units, 100 epochs, learning rate 0.001, weight decay 0.0001, and Efron ties. The discrete neural model uses 4 or 8 hidden units, eight intervals, weight decay 0.01, and at most 1,000 optimization iterations. All fixed and selected settings are written to `selected_settings.csv`. The ranking, hybrid and IPCW grids use both kernels and $\lambda \in \{10^{-5}, \ldots, 10^{1}\}$. Hybrid weights are $0.25, 0.5, 0.75$; IPCW horizons are 90 and 180 days, with epsilon 0 or 0.02. Their optimization budget is at most 5,000 iterations and they retain the default stationarity tolerance. Each identifier selects its kernel within the same three-fold procedure. The horizon is a candidate target, not an estimate derived from the test outcomes; the reported IPCW fits use marginal censoring weights and retain the associated assumptions.

5.2. Fitting and interpreting the obtained results

After creating `sim_train` and `sim_test` with the replication script, elastic-net Cox estimation is performed as follows:

```
R> f <- Surv(time, event) ~ temperature + power_hours + load + design
R> tuned <- surv_tune(f, sim_train, method = "cox_elasticnet",
+    grid = list(lambda = c(0.001, 0.01), alpha = c(0.25, 0.75)),
+    folds = 3, seed = 20261021)
R> tuned$best_params
R> surv_score(tuned, sim_test)
R> surv_predict(tuned, sim_test[1:2, ], type = "survival",
+    times = c(30, 90))
```

The calling sequence is retained for other methods by changing the identifier and appropriate grid. A median request can be made for `aft` or `xgboost_aft`; it is intentionally unavailable for `cox_elasticnet` in this version.

From Table 2, we observe that the Gehan AFT and Cox configurations produce similar test concordance, approximately 0.685–0.686. The MTLR result is 0.687, and the single tree returns 0.630. The ranking, hybrid and IPCW SVMs obtain 0.687, 0.686 and 0.687, respectively. These scores are close to those of the regression references; the additional objectives do not establish a uniform gain in this generated example. Differences between several regression configurations are small relative to their conditional standard errors. These observations concern one training sample, one test sample, and the specified grids; they do not establish a preferred family under other nonlinearities, censoring patterns, or sample sizes.

The original censored-regression SVM gives test concordance 0.688, and native neural Cox

Method	CV C	Test C	SE
Parametric AFT	0.687	0.686	0.011
Gehan AFT LASSO	0.689	0.685	0.011
Gehan AFT ridge	0.688	0.686	0.011
Gehan AFT elastic net	0.689	0.685	0.011
Cox	0.686	0.685	0.011
Cox LASSO	0.686	0.685	0.011
Cox ridge	0.686	0.685	0.011
Cox elastic net	0.686	0.685	0.011
Survival tree	0.600	0.630	0.012
Random survival forest	0.663	0.679	0.011
Cox gradient boosting	0.633	0.678	0.011
XGBoost Cox	0.634	0.680	0.011
XGBoost AFT	0.630	0.677	0.011
MTLR	0.688	0.687	0.011
Censored SVM	0.688	0.688	0.011
Spline Cox	0.661	0.683	0.011
Flexible parametric	0.685	0.685	0.011
Componentwise CoxBoost	0.686	0.685	0.011
Ranger forest	0.634	0.673	0.011
Oblique forest	0.668	0.682	0.011
Native neural Cox	0.659	0.665	0.012
Discrete neural survival	0.639	0.659	0.012
Ranking SVM	0.687	0.687	0.011
Hybrid SVM	0.688	0.686	0.011
IPCW SVR	0.684	0.687	0.011

Table 2: Generated example: mean tuning concordance, independent test concordance, and conditional infinitesimal-jackknife standard error. Time and memory are measured separately in Section 8.

gives 0.665. The latter is not uniformly superior to the linear models: the generated mechanism in Equation 54 is a log-normal AFT mechanism, and the neural Cox model still imposes proportional hazards. A flexible covariate map does not remove that restriction. There are 0 failed candidate-fold fits under these settings. Warnings are recorded separately from errors, since an accepted fit can still produce a warning from its engine.

6. A historical reliability application

6.1. Cohort construction and descriptive analysis

We consider the public Backblaze Drive Stats data for 2013 (Backblaze 2026). The source comprises 266 daily snapshots from April 10 to December 31, containing 5,091,501 records. The processed cohort has one row per drive serial number. If d_{i0} is the first recorded date and d_{i1} is the failure or final recorded date, the observed duration is $Y_i = d_{i1} - d_{i0}$ elapsed days. The event indicator identifies a recorded failure; otherwise the drive is treated as right-censored. Predictions are made after the first snapshot, conditional on being operational at that snapshot.

Sample	Drives	Failures	Censored	Censored (%)
Cohort	29,058	718	28,340	97.53
Training	23,246	574	22,672	97.53
Test	5,812	144	5,668	97.52

Table 3: Backblaze cohort and event-stratified random 80/20 split. Durations begin at first recorded observation, not at installation.

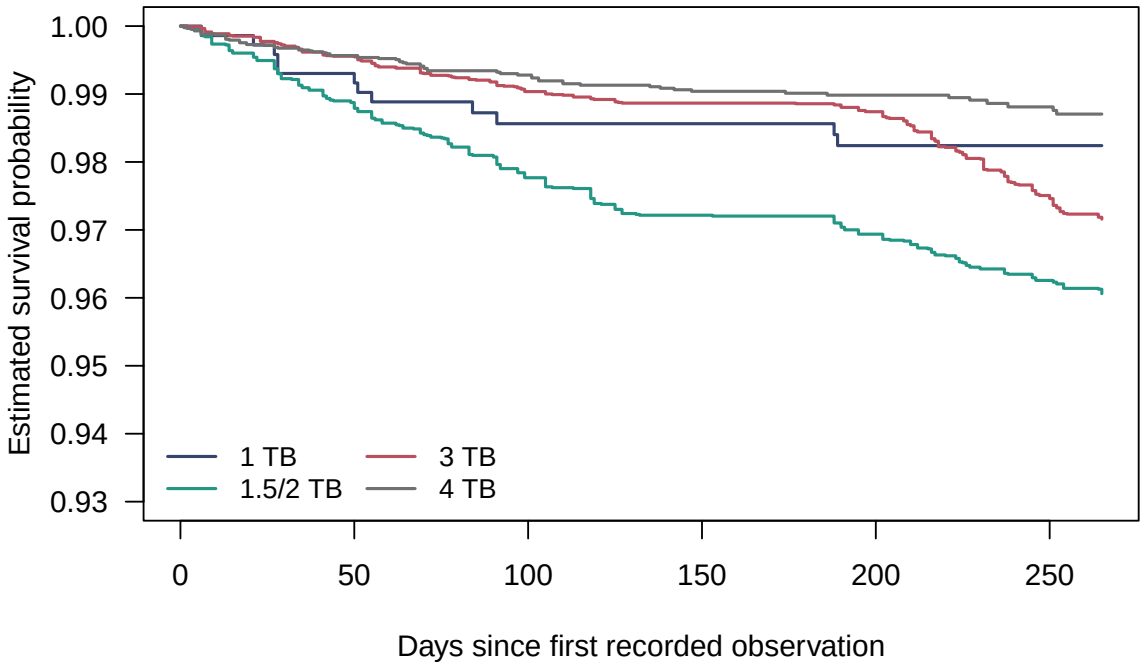


Figure 1: Kaplan–Meier estimates by drive capacity in the complete 2013 cohort. The vertical scale is restricted to make the differences visible. The 1.5/2 TB group pools both nominal capacities. Capacity groups also differ in other characteristics and calendar entry.

The source contains 29,072 drives and 724 recorded failures. Six drives fail at their first snapshot, and eight others have no positive follow-up after entry. These 14 histories remain in the source file with their exclusion reasons, but cannot enter the risk set for subsequent failure. The analysis therefore includes 29,058 drives, 718 failures, and 28,340 censored observations (97.53% censoring). Both the median and maximum observed duration are 265 days. These exclusions define the prediction problem; they do not reduce the sample for SVM computation. The median observed duration should not be confused with the median failure time. Table 3 describes the analysis partition.

The Kaplan–Meier curves in Figure 1 show small failure probabilities. Capacity groups also differ in manufacturer, model, operating history, and entry date, so their contrasts are not isolated physical effects. The rounded level stored as `2 TB` combines 3,102 nominal 1.5 TB drives (264 failures) and 5,198 nominal 2 TB drives (55 failures). We retain this encoding in fitting and label it 1.5/2 TB; the supplied byte-level table records the pooling. Independent censoring remains an assumption.

Predictor	Definition
Manufacturer	Factor derived from recorded model.
Capacity group	Factor: 1, pooled 1.5/2, 3, or 4 TB.
`log_power_hours`	$\log(1 + \text{SMART}_9)$ at the first snapshot.
`temperature`	Raw SMART 194 at the first snapshot.
`log_reallocated`	$\log(1 + \text{SMART}_5)$ at the first snapshot.
`pending`	Indicator that entry-snapshot SMART 197 is positive.
`entry_day`	Days between first drive observation and April 10, 2013.

Table 4: Predictor definitions. Selected SMART measurements are taken from the exact first snapshot, never from a later nonmissing value. The retained baseline date permits a direct availability audit.

The seven predictors are defined in Table 4. Diagnostic variables come from self-monitoring, analysis, and reporting technology (SMART) attributes. The transformation $\log(1+x)$ retains zeros and compresses a right-skewed count or operating-time scale. It does not make the variable normally distributed, and predictor normality is not required by these models. In this cohort, median power-on hours are 7,469 (interquartile range 869–17,250), and median recorded temperature is 26 (interquartile range 23–29). Both sector-count variables have zero at their first and third quartiles, with large positive maxima. Manufacturer and model-specific interpretation of raw SMART values remains relevant.

All four SMART measurements are available at the first snapshot. No later value or imputation is used. The script saves baseline dates, eligibility, counts, and missingness for verification. The quantity being predicted is survival after first appearance, not lifetime from installation or recurrent failure. Confirming that predictors were available at entry does not make this historical random split a prospective validation.

6.2. Training, tuning, and held-out comparison

The 80/20 split is made by drive, stratified by event, with seed 20261022. The training sample has 23,246 drives and 574 failures; the test sample has 5,812 drives and 144 failures. Test rows do not enter selection. All candidates use the same three training folds with seed 20261023. The split is random, not chronological, and does not assess transfer to a later year or an unseen model. Grids are those in Section 5. For example, the forest is fitted by

```
R> form <- Surv(time_days, event) ~ manufacturer + capacity_group +
+    log_power_hours + temperature + log_reallocated +
+    pending + entry_day
R> forest <- surv_tune(form, train, method = "rsf",
+    grid = list(min_node = c(15L, 50L)),
+    params = list(trees = 200L, threads = 1L),
+    folds = 3, seed = 20261023)
R> surv_score(forest, test)

[1] 0.8558433
```

The application fits 20 configurations on the entire training sample, including native neural Cox and separate linear and additive searches for each of the four SVM objectives. The six

Method	CV C	Test C	SE
Parametric AFT	0.795	0.823	0.017
Cox	0.795	0.824	0.017
Cox LASSO	0.796	0.829	0.017
Cox ridge	0.798	0.824	0.017
Cox elastic net	0.799	0.826	0.017
Survival tree	0.792	0.814	0.016
Random survival forest	0.846	0.856	0.015
Cox gradient boosting	0.812	0.829	0.018
XGBoost Cox	0.834	0.837	0.019
XGBoost AFT	0.829	0.824	0.018
MTLR	0.792	0.813	0.017
SVR (linear)	0.493	0.491	0.015
SVR (additive)	0.717	0.695	0.020
Native neural Cox	0.795	0.813	0.017
Ranking SVM (linear)	0.800	0.818	0.017
Ranking SVM (additive)	0.827	0.833	0.018
Hybrid SVM (linear)	0.799	0.818	0.017
Hybrid SVM (additive)	0.827	0.833	0.018
IPCW SVR (linear)	0.776	0.796	0.017
IPCW SVR (additive)	0.789	0.808	0.017

Table 5: Backblaze fits. CV is the largest eligible three-fold mean concordance. Test SE is the conditional infinitesimal-jackknife standard error. The training and test samples are the same for every method.

additional searches use the same lambda, hybrid-weight, horizon and epsilon grids as the controlled example, with the kernel fixed within each search. Failed candidate-fold fits are recorded and cannot be selected. Test outcomes are not used to modify the grids. However, this historical split was inspected during earlier method development. The extended comparison is therefore exploratory, not a new external validation on previously unseen data. For the original censored absolute-loss regression, both kernels use Clarabel and separate seven-cost searches. The three additional SVM objectives use L-BFGS-B and their declared lambda grids on the same event-stratified folds. The comparison does not use a subset enriched with failures. The three Gehan AFT configurations are fitted only to the generated sample because of their computational cost; their omission here does not show inferior prediction. The remaining reference methods are also illustrated in the generated example, rather than in a full large-data benchmark.

From Table 5, we observe that RSF has the largest mean tuning concordance (0.846) and test concordance (0.856) among the fitted configurations. XGBoost Cox gives test concordance 0.837; additive ranking and hybrid SVMs give 0.833, followed by Cox gradient boosting (0.829). The ordinary Cox and selected Weibull AFT fits give approximately 0.824 and 0.823, respectively. These results concern discrimination within this historical split. The conditional standard errors are not a paired test of RSF against XGBoost Cox and exclude split, training, and tuning variability.

Native neural Cox gives test concordance 0.813. The original linear SVR is close to 0.5 after numerical ties are respected; ordering solver-scale perturbations would produce artificial discrimination. The SVM objectives are compared below, separately from the numerical

experiments in Section 8.

The original linear SVR has the same validation concordance, 0.4931, throughout its cost grid at the declared resolution, so the first entry, 0.001, is retained. Additive SVR selects cost 0.1 with CV concordance 0.7169. The complete cost paths are supplied in `svm_tuning.csv`. There are 5 failed candidate-fold fits in the real-data search. The log-normal and log-logistic AFT candidates fail the coefficient/scale validation in two and three folds, respectively, and are excluded. The selected Weibull AFT and all penalized Cox candidate-fold fits succeed. All candidate errors remain in the replication outputs.

Ordinary Cox fitting also warns of possible infinite coefficients for sparse factor categories in training folds. Predictions remain finite, but the warnings are saved in `engine_warnings.txt`, and these coefficients are not interpreted as stable hazard ratios. A warning alone differs from a failed fold: selection excludes errors, failed numerical checks, and nonfinite concordance, but not every engine warning.

RSF selects node size 50 with 200 trees and three candidate predictors. Both XGBoost objectives select 200 trees; AFT selects normal errors with scale one. Native neural Cox selects 16 hidden units. All selected parameters are supplied in `selected_settings.csv`; Table 6 gives the additional SVM settings. Because grids differ, Section 8 measures time and memory at fixed settings rather than comparing unequal search budgets.

Comparing SVM learning targets

The original regression and the three additional objectives use identical covariates and training folds. Their targets nevertheless differ. Ranking linear obtains test concordance 0.818, compared with 0.491 for the original linear regression. Ranking additive obtains 0.833, compared with 0.695 for its absolute-loss counterpart. Hybrid linear and additive give 0.818 and 0.833, whereas IPCW linear and additive give 0.796 and 0.808. The RSF point estimate remains larger, at 0.856. These results show why the loss and the kernel should be considered separately: poor discrimination under the absolute loss does not imply poor discrimination for every survival SVM.

Objective	Kernel	λ	η	τ	ϵ
Ranking	linear	1e-05	–	–	–
Ranking	additive	1e-03	–	–	–
Hybrid	linear	1e-02	0.75	–	–
Hybrid	additive	1e-03	0.75	–	–
IPCW	linear	1e+00	–	180	0
IPCW	additive	1e-01	–	90	0

Table 6: Training-CV-selected settings for the additional SVM objectives. The six searches use all training drives; the test outcomes do not determine these settings. The horizon is in days and epsilon is dimensionless.

For example, the additive ranking search and its held-out score are obtained by

```
R> ranking <- surv_tune(form, train, "svm_rank",
+    grid = list(lambda = 10^(-5:1)),
+    params = list(kernel = "additive", max_iter = 5000L),
+    folds = 3, seed = 20261023)
R> ranking$best_params$lambda
R> surv_score(ranking, test)

[1] 0.001
[1] 0.8327246
```

The comparable-pair calculation retains 11,047,936 training pairs. For the paired ranking-additive minus RSF difference, 1,000 test bootstrap resamples give -0.0231, with percentile interval $[-0.0403, -0.0069]$. This comparison conditions on the fitted and selected models. It excludes training and tuning uncertainty and does not provide simultaneous inference across the catalogue. The interval lies below zero, favoring RSF in this particular conditional comparison.

Reducing optimization tolerance from 10^{-6} to 10^{-7} changes none of the six three-decimal concordances; the largest absolute change is below 10^{-6}. Conditional Cox censoring fits fail support checks at both selected IPCW horizons and are rejected without clipping weights or removing drives. Entry date limits follow-up. A positive marginal $\widehat{G}$ therefore does not establish conditional support for drives entering late. The marginal IPCW results illustrate prediction under the stated assumptions; they do not show that restricted means can be identified for every profile or that predicted probabilities are calibrated.

6.3. Prediction for additional covariate profiles

We consider three hypothetical 3 TB Hitachi profiles entering on the first day of the 2013 window. Power-on hours are 5,000, 15,000, and 30,000; temperatures are 25, 30, and 35; reallocated-sector counts are 0, 10, and 100; pending-sector indicators are 0, 0, and 1, respectively. These are constructed profiles, not additional observed failures. Several predictors change together, so their contrasts are not isolated covariate effects. Factor levels are those learned from training:

```
R> S <- surv_predict(forest, new_drives, type = "survival",
+    times = c(90, 180))
R> round(1 - S, 4)

       t=90  t=180
[1,] 0.0008 0.0011
[2,] 0.0097 0.0163
[3,] 0.1148 0.1387
```

The matrix contains modeled failure probabilities. For example, profile 3 has a modeled failure probability of approximately 13.87% by 180 days under this fitted forest. A call with

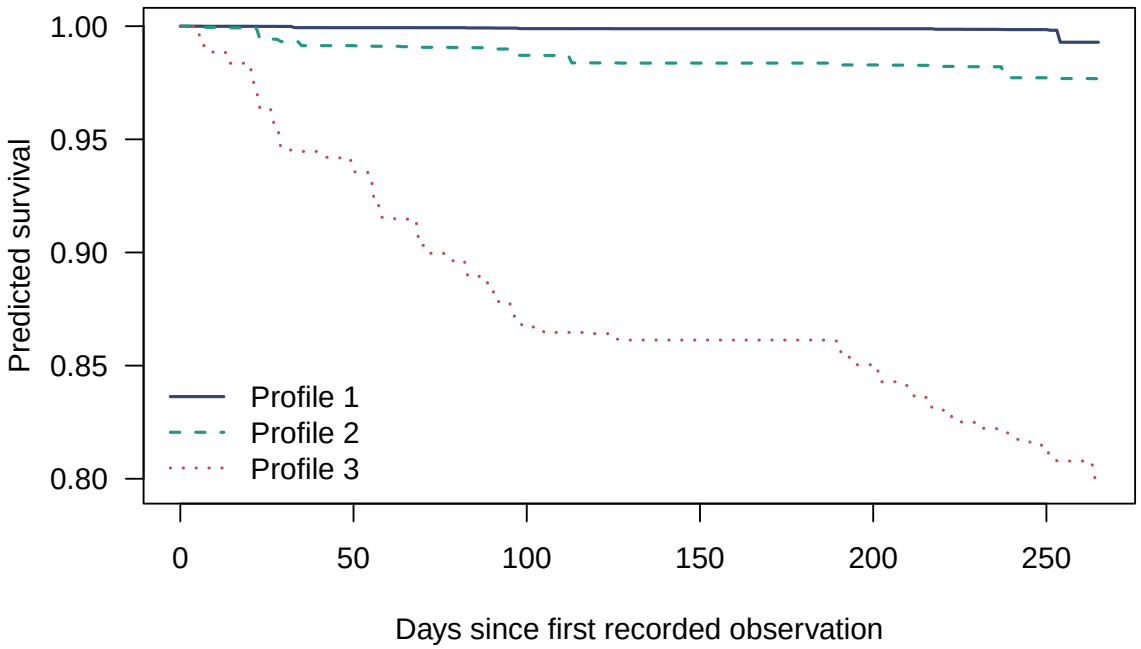


Figure 2: Fitted RSF survival curves for three constructed profiles. These are outputs of a historical model, not externally validated predictions for current hardware or Backblaze operations.

`type = "risk"` instead returns mortality scores; these must not be substituted for the matrix probabilities.

Figure 2 and Table 7 compare fitted curves and probabilities. Their differences require calibration assessment, not concordance alone. A parametric median beyond 265 days depends on extrapolation.

Using the training-score quartiles to group test scores gives four risk groups. The lowest has four failures among 1,501 drives, and the highest has 117 among 1,421. These counts are descriptive, not Kaplan–Meier probabilities or calibration estimates. A maintenance policy would also require replacement costs, failure consequences, and prospective evaluation.

Method	Profile	$\widehat{F}(90)$	$\widehat{F}(180)$
Random survival forest	1	0.0008	0.0011
Random survival forest	2	0.0097	0.0163
Random survival forest	3	0.1148	0.1387
XGBoost Cox	1	0.0017	0.0024
XGBoost Cox	2	0.0025	0.0037
XGBoost Cox	3	0.1209	0.1698
XGBoost AFT	1	0.0002	0.0019
XGBoost AFT	2	0.0003	0.0031
XGBoost AFT	3	0.0578	0.1893
MTLR	1	0.0066	0.0107
MTLR	2	0.0141	0.0225
MTLR	3	0.0945	0.1366
Native neural Cox	1	0.0065	0.0093
Native neural Cox	2	0.0124	0.0175
Native neural Cox	3	0.0441	0.0622

Table 7: Modeled failure probabilities for the constructed profiles under five adapters returning survival curves. Concordance does not establish that these probabilities are calibrated.

6.4. Probability assessment and conditional calibration

Harrell concordance remains the tuning criterion. We next evaluate the probabilities returned by models that provide survival curves. The archive ends on December 31, so a drive entering near that date cannot be observed for 180 days. We therefore define the probability-assessment population as drives entering by July 4, 2013. This gives 19084 reference training drives and 4751 held-out drives, with 137 recorded test failures. Event models remain fitted to all 23,246 eligible training drives. Only the assessment population and its censoring reference change.

A reverse-KM censoring model uses prespecified 30-day entry bands, avoiding a proportional-hazards assumption for calendar entry. Withdrawal before failure must still be conditionally independent within each band, and the reference censoring distribution must apply to the assessment sample. Positive estimated censoring probabilities do not verify these assumptions. Brier scores use Equation 45; IBS is the normalized trapezoidal integral over 30, 60, 90, 120, 150 and 180 days. No test-driven retuning or censoring-weight clipping is performed.

From Table 8, we observe an IBS of 0.01180 for RSF. XGBoost Cox gives 0.01185, whereas XGBoost AFT has the smallest 180-day Brier score, 0.01515. Thus the ranking depends on the prediction target and criterion. Small absolute errors partly reflect the rarity of failures and should not be interpreted as sufficient evidence of calibration. These point estimates are not a paired significance test or evidence of prospective calibration.

Figure 3 uses the normalized IPCW ratio in Equation 48, not a marginal within-group KM. At 180 days, the lowest-survival group has mean predicted survival 0.9512 and weighted observed survival 0.9434 (interval 0.9278–0.9567). The highest-survival group has no failures by either horizon, and another 90-day group has only two failures. Their `low_support` flags remain in the exported table. The fixed-fit bootstrap cannot infer an unobserved rare-event tail or account for model fitting. Calibration is therefore presented as a qualified diagnostic, not as confirmation of a maintenance decision rule.

Method	Brier(90)	Brier(180)	IBS(30–180)
Parametric AFT	0.01019	0.01534	0.01193
Cox	0.01013	0.01544	0.01191
Cox LASSO	0.01035	0.01579	0.01216
Cox ridge	0.01022	0.01556	0.01200
Cox elastic net	0.01026	0.01562	0.01204
Random survival forest	0.01004	0.01537	0.01180
XGBoost Cox	0.01002	0.01547	0.01185
XGBoost AFT	0.01023	0.01515	0.01188
MTLR	0.01043	0.01596	0.01225
Native neural Cox	0.01087	0.01655	0.01266

Table 8: Native probability error in the prespecified early-entry target. Lower values are preferred. All rows use the same reference censoring fit; risk-only learners are not assigned invented probability curves.

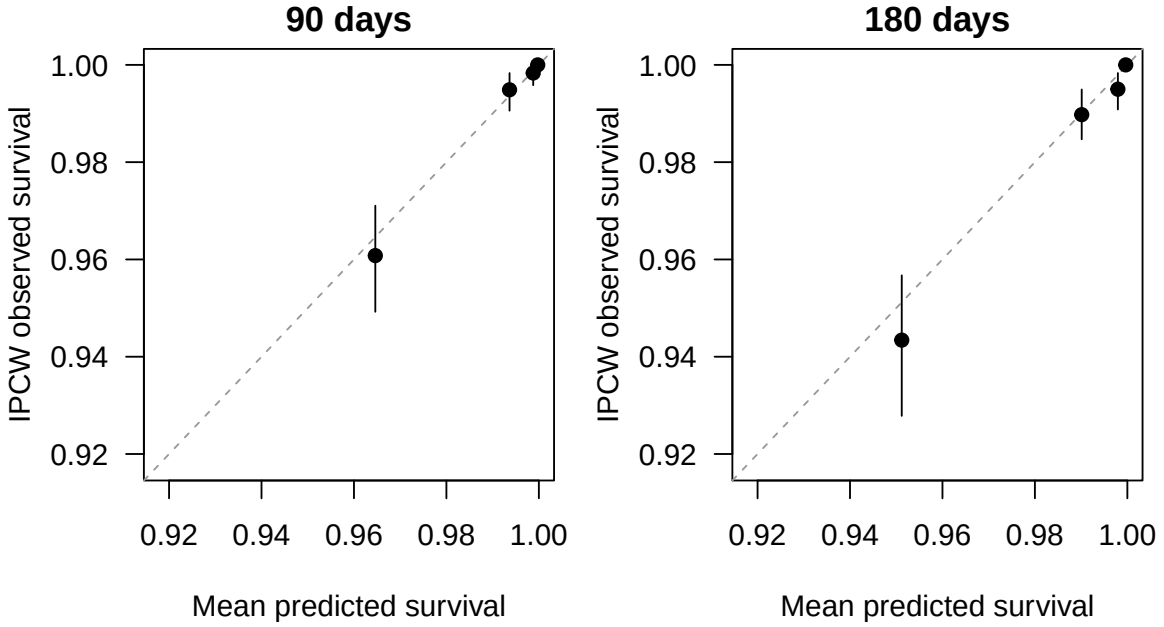


Figure 3: RSF grouped calibration at 90 and 180 days in the early-entry test target. Groups use predicted-survival midranks. Vertical intervals are 95% percentiles from 200 within-group bootstrap repetitions with the fitted event model, censoring model and groups fixed. The diagonal denotes agreement. The high-survival groups include sparse or zero events; degenerate bootstrap intervals at one do not imply certainty.

7. Extended workflow examples

7.1. Paired comparison, calibration and stacking

The following Cox–RSF stack retains all 23,246 training drives. RSF node size is selected in three inner folds; the weights use three OOF folds. Marginal reverse-KM weights require

marginal independent censoring, a limitation in this administratively censored cohort. The selected weights are 0.396 for Cox and 0.604 for RSF. The OOF objective is 0.00950, not a test result.

```
R> stack_fit <- surv_stack(form, train, methods = c("cox", "rsf"),
+    grids = list(rsf = list(min_node = c(15L, 50L))),
+    params = list(rsf = list(trees = 200L, threads = 1L)),
+    times = c(30, 90, 180), folds = 3, inner_folds = 3,
+    seed = 20261104)
R> surv_compare(list(cox = real_models$cox, stack = stack_fit), test,
+    reference = "cox", n_boot = 1000, seed = 20261104)
R> surv_calibrate(stack_fit, prob_test, times = 180, type = "metrics",
+    censoring_model = prob_g, n_boot = 500, seed = 20261104)
```

Tables 9 and 10 use the original full test sample for concordance and the prespecified early-entry target for probability assessment, respectively. The latter retains the entry-band censoring reference. The stack refits its components and uses negative restricted mean as its score; the standalone RSF uses mortality. Their concordance difference therefore does not isolate the effect of combining identical fitted models. The stack gives $C = 0.838$, below the standalone RSF. Its paired difference from Cox is 0.014, with an interval including zero. At 180 days, its Brier score is 0.01510, compared with 0.01544 for Cox and 0.01537 for RSF. Calibration intervals cover the ideal intercept and slope for both reported models; this does not establish perfect calibration.

Model	Test C	ΔC vs. Cox	95% interval
RSF	0.856	0.032	[0.010, 0.051]
Additive ranking SVM	0.833	0.009	[-0.013, 0.028]
Additive hybrid SVM	0.833	0.009	[-0.012, 0.028]
Cox–RSF stack	0.838	0.014	[-0.001, 0.030]

Table 9: Paired comparison on all 5,812 test drives. Differences are relative to Cox; intervals use 1,000 common test-case bootstrap samples, with fitted models fixed.

Model	Intercept [95% interval]	Slope [95% interval]	Brier(180)
Cox	0.158 [-0.090, 0.353]	1.057 [0.741, 1.339]	0.01544
Cox–RSF stack	0.206 [-0.030, 0.399]	1.132 [0.932, 1.315]	0.01510

Table 10: IPCW calibration at 180 days in the existing probability target. Intervals use 500 test-case samples with event/G fits fixed. Intercept and slope have ideal values zero and one.

7.2. Held-out interpretation and remaining duration

Table 11 uses all 5,812 test drives, five common permutations and seed 20261024. Manufacturer has the largest importance for Cox, RSF and neural Cox. For RSF, its concordance decrease is 0.144, followed by operating hours (0.078) and capacity (0.074). Negative temperature importance for neural Cox means permutation improves ranking in this sample, not a protective physical effect.

Predictor	Cox	RSF	Neural Cox
manufacturer	0.1905 (0.0256)	0.1442 (0.0144)	0.1361 (0.0204)
capacity_group	0.0325 (0.0066)	0.0745 (0.0085)	0.0231 (0.0084)
log_power_hours	0.0325 (0.0063)	0.0776 (0.0166)	0.0070 (0.0047)
temperature	0.0010 (0.0022)	0.0130 (0.0072)	-0.0048 (0.0043)
log_reallocated	0.0383 (0.0034)	0.0464 (0.0040)	0.0485 (0.0048)
pending	0.0133 (0.0018)	0.0075 (0.0020)	0.0095 (0.0029)
entry_day	0.0056 (0.0087)	0.0274 (0.0074)	0.0000 (0.0060)

Table 11: Decrease in held-out Harrell concordance after permuting each original predictor. Parentheses contain permutation SD, not confidence intervals. All test rows are retained.

Figure 4 combines RSF importance with ALE on $F(180 \mid x)$ using assessment quantile bins. Manufacturer-dependent SMART encoding limits physical interpretation of temperature; these effects are not causal.

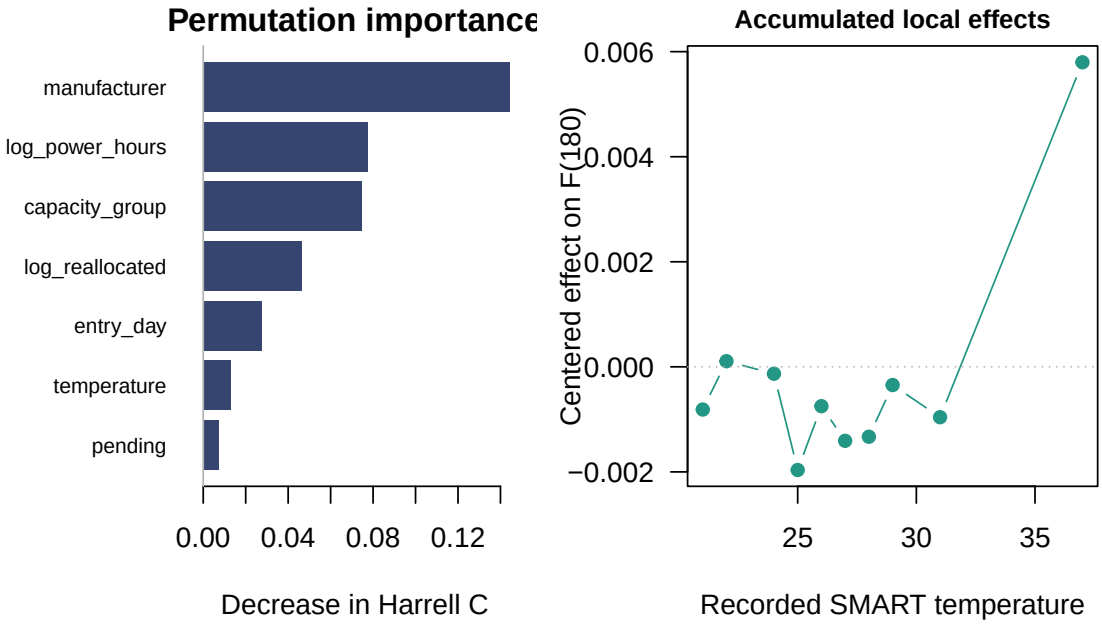


Figure 4: RSF interpretation on every held-out drive. Left: mean permutation importance. Right: centered binwise local effects on 180-day failure probability. No inferential uncertainty band or causal interpretation is added.

Table 12 conditions on survival to model day 30, using grid spacing at most 0.3 day. Profile 3 has next-150-day failure probability 0.0888, restricted remaining mean 140.65 days, and a 1% quantile at 20.1 days. Profiles 1 and 2 do not reach that quantile; none reaches its median. These fixed-covariate summaries do not estimate an unrestricted lifetime.

7.3. A controlled conditional-censoring example

To illustrate the role of the censoring model separately from the historical cohort, we generate independent training and test samples of size 2,000 each, using seed 20261025. Let $X_1 \sim$

Profile	$F_a(60)$	$F_a(150)$	$\mu_{a,150}$	$q_{a,.01}$
1	0.0007	0.0010	149.88	Not reached
2	0.0030	0.0096	149.11	Not reached
3	0.0636	0.0888	140.65	20.1

Table 12: RSF summaries conditional on survival to day 30 since first recording. Durations and restricted means are in days. A quantile marked "Not reached" has no crossing within the largest future horizon of 150 days.

$\mathcal{U}(-1,1)$ and $X_2 \sim \text{Bernoulli}(1/2)$ independently, and generate

$$\begin{aligned} T \mid x &\sim \text{Exp}\{0.02\exp(0.8x_1 - 0.3x_2)\}, \\ C^* \mid x &\sim \text{Exp}\{0.03\exp(x_1 + 0.2x_2)\}, \qquad C = \min(C^*, 40), \end{aligned} \tag{56}$$

with independent exponential draws given x. Rates, rather than mean parameters, are shown. The training censoring percentage is 73.0%. Before the administrative endpoint, the Cox censoring specification is correct; the evaluation horizons 10 and 15 are strictly earlier than that endpoint.

We fit Cox to the observed training data and evaluate the same predictor using complete generated test times, marginal training reverse KM, and conditional training Cox G. Complete outcomes are available only in this generated example.

Criterion	Complete outcomes	Marginal G	Conditional G
Harrell C	0.6230	0.6340	0.6340
Pair-weighted $C(15)$	0.6428	0.6438	0.6488
AUC(10)	0.6573	0.6923	0.6763
Brier(10)	0.1384	0.1369	0.1439

Table 13: One controlled realization under covariate-dependent censoring. Every column evaluates the same event-model predictions. Complete outcomes provide a finite-sample reference, not a population value.

Harrell concordance is unchanged between observed-outcome columns because it does not use G. Conditional AUC is 0.6763, compared with marginal 0.6923 and complete-outcome 0.6573; conditional Brier is 0.1439, compared with 0.1369 and 0.1384, respectively. Conditional weighting is therefore not uniformly closer to the complete-data reference in this realization. The weights rely on the censoring model, not a guarantee of improvement in every sample. Misspecification or limited follow-up can increase instability.

7.4. Full-training parallel and resume comparison

Sequential and two-worker PSOCK searches use all 23,246 training drives, linear SVM with Clarabel, the same two costs, three folds and seed 20261023. Elapsed time includes startup, checkpoint access, candidate fits and final refitting, but excludes test prediction. A third call resumes the completed parallel search.

Serial and parallel runs select the same grid row, with maximum fold-score difference 0 and prediction difference 0. Sequential execution takes 6.00 seconds, whereas two workers take

Run	Seconds	New fits	Reused folds	Refit reused
Serial	6.00	6	0	No
Two workers	19.73	6	0	No
Completed resume	1.78	0	6	Yes

Table 14: Measured local execution of the same full-training SVM search. "New fits" counts candidate-fold fits, excluding the final refit. The resume row reuses a completed search; it is not a partial-interruption timing.

19.73 seconds, or 3.29 times as long. Reusing the completed search takes 1.78 seconds and avoids all six fold fits and the final refit. A separate test removes one fold record and its saved final fit, then verifies that five folds are reused and one is recomputed. Total PSOCK memory was not measured. Parallel execution need not accelerate a small search.

8. Numerical verification and computational experiments

8.1. Protocol and interpretation

We next examine numerical agreement, time and memory without reselecting models. The scripts `verify_svm.R` and `benchmark.R` use the same training and test drives as Section 6.

Measurements use Windows 11 x64, R 4.6.0, the default BLAS, and an Intel Core Ultra 7 155H with approximately 15.6 GiB usable physical memory. The original regression SVM kernels use cost 0.1. The six ranking, hybrid and IPCW fits use the settings selected by training CV in Table 6. Different objectives are not claimed to be computationally equivalent estimators. Neural Cox uses 16 hidden units, 100 epochs, learning rate 0.001, decay 0.0001, and seeds 901, 902 and 903, separately under Efron and Breslow ties. These fixed configurations are distinct from validation-selected configurations in the predictive comparison.

Each repetition starts a fresh R process. Fitting seconds exclude startup, while peak working set includes libraries, data, fitting and prediction. The launcher reads the process peak and samples private bytes every 100 milliseconds. MiB denotes 2^{20} bytes. These measurements are not fitted-object sizes or baseline-subtracted allocations. A 900-second or 3,500-MiB private-memory limit is recorded as unsuccessful termination, never as a successful fit. No cross-platform speedup or GPU comparison is claimed.

8.2. Independent numerical verification

Twelve 24-observation test samples per kernel cover low and high event fractions, all-observed failures, repeated predictor values and collinearity. For each sample, we independently assemble the dense kernel and box-constrained dual in Equation 35. This is compared with the sparse primal's objective, rather than with a conditioned Hessian from another program. A second check reconstructs predictions from the complete kernel and recovered dual coefficients. Table 15 reports the largest differences.

Package tests also compare Clarabel and OSQP, check prediction support, rare levels, duplicates, time units, saved tuning and numerical ties. Neural Cox tests enumerate tied risk sets and compare analytic gradients with finite differences. These small samples do not replace

Check	Maximum difference	Tolerance
linear primal/dense dual (relative)	3.14e-11	10^{-7}
linear centered representer	8.91e-12	10^{-6}
additive primal/dense dual (relative)	4.40e-11	10^{-7}
additive centered representer	4.35e-13	10^{-6}

Table 15: Independent sparse-primal/dense-dual checks across twelve fixtures for each kernel. All calculations use double precision.

Configuration	R	Seconds	MiB	Max MiB	Test C
Linear SVM, Clarabel	3	1.30	288.3	288.7	0.4915
Additive SVM, Clarabel	3	2.00	251.7	251.8	0.6952
Neural Cox, Efron	3	2.14	249.3	249.3	0.8066
Neural Cox, Breslow	3	2.14	249.0	249.4	0.8066
Ranking SVM, linear	3	0.37	230.8	230.9	0.8176
Ranking SVM, additive	3	1.81	230.3	231.2	0.8327
Hybrid SVM, linear	3	0.42	230.9	230.9	0.8180
Hybrid SVM, additive	3	2.04	232.0	232.5	0.8328
IPCW SVR, linear	3	0.18	228.6	232.8	0.7959
IPCW SVR, additive	3	0.27	231.4	232.1	0.8077

Table 16: Full-training measurements with **survlearning** 0.3.5. R is the number of fresh processes. Seconds, MiB and test concordance are medians; Max MiB is the largest process peak. The original regression SVMs use cost 0.1; the other SVMs use the training-selected settings in Table 6. All analysis rows are retained.

application rows.

For the additional SVM objectives, independent quadratic programs enumerate the ranking pairs and assemble the hybrid and positive-epsilon IPCW losses for both kernels. The fitted objectives and predictions are compared with these references. Zero-epsilon IPCW is also checked against weighted normal equations. Finite differences verify the analytic gradients, and a separate invariance test adds common score offsets as large as 10^{12} without changing the ranking loss or gradient. These checks distinguish the statistical objective from a particular optimizer implementation.

8.3. Time and memory

Table 16 reports medians across three fresh processes for each configuration. The original regression SVMs use the default training-fixed prediction resolution and return independent primal–dual gaps. Neural rows compare different tie likelihoods; similar concordance would not establish mathematical equivalence.

These results document resource requirements on one machine, not superiority over every external solver. The additional SVM fits take 0.18–2.04 seconds at the selected settings, with median process peaks of 228.6–232.0 MiB. These are single-fit measurements, not the total cost of tuning. A dense full-cohort kernel is not allocated merely to obtain a memory comparison; a quadratic-storage calculation would not be a measured process peak. A completed neural epoch budget is also not a convergence certificate. The exported files retain versions, sample

sizes, numerical gaps, concordance, elapsed times and process termination status.

Accurate minimization does not necessarily give good risk ordering. Under heavy censoring, many predictions can be equal to numerical precision. The grid in Appendix A prevents small solver differences from being counted as discrimination. Its resolution is fixed before evaluation, not selected from test outcomes, so nearly tied linear-SVR scores are not given an artificially favorable ordering.

8.4. Matched software workflows and split sensitivity

To assess the interface, we fit ordinary Cox and Weibull AFT through **survlearning** and **censored** 0.3.5 with **parsnip** 1.6.1 (Kuhn and Vaughan 2026). Both use **survival** 3.8-12, the same formula, full training/test samples and three identical assessment folds. Cox uses Efron ties. Neither workflow performs a hyperparameter search here.

The tested **censored** `linear_pred` output points toward longer survival for both models, so its sign is reversed. Survival predictions arrive as a nested tibble, rather than our matrix. Centered scores agree exactly. At 30, 90 and 180 days, maximum survival differences are 5.9×10^{-15} for Cox and 5.6×10^{-17} for AFT. Both interfaces give concordances 0.8240 and 0.8228, respectively. This verifies shared engines, not estimator superiority. Our interface has fewer conventions; **censored** connects to broader **tidymodels** tools.

Split seeds 20261022, 20261028 and 20261029 provide a sensitivity check with unchanged sample/event counts and no retuning. Fixed settings are ordinary Cox, RSF with 200 trees, node size 15 and three candidate predictors, and neural Cox with 16 units and 100 Efron epochs. Their concordance ranges are 0.7971–0.8254, 0.8475–0.8535 and 0.7887–0.8127; medians are 0.8240, 0.8509 and 0.8030. These describe overlapping historical splits, not independent replications or prospective performance. Complete interface and split results are supplied in `framework_comparison.csv` and `split_sensitivity.csv`.

9. Reproducibility

The entry script `code.R` runs all analyses, extensions and benchmarks from the beginning, without loading saved models. Outputs include splits, fold results, settings, warnings, tables, figures and software versions. The supplied CSV is sufficient; `prepare_data.R` reconstructs it from the annual Backblaze archive and records eligibility, dates and checksums.

The experiments use the supplied **survlearning** 0.3.5 source archive and R 4.6.0 on Windows. The entry script checks the archive checksum, and the accompanying version table identifies the installed dependencies. No script installs packages or downloads data during analysis.

`benchmark.ps1` measures process memory on Windows; other systems leave memory unmeasured. Package tests check engine agreement, ties, extreme predictors, preprocessing and censoring fits within folds, serialization and recovery. Independent references include quadratic programs, finite differences, weighted normal equations and explicit pair sums. Checks on other operating systems must use the same source archive.

10. Discussion

In this paper, we presented a common interface for fitting, selecting, and evaluating survival learning methods for right-censored data. The contribution is a reproducible implementation of established methods, not a new loss. In the Backblaze application, ranking and hybrid SVMs give better discrimination than absolute-loss regression using the same covariates and complete training sample. The differences are much smaller in the generated example. Thus, the comparison does not establish a general ordering of the methods. IPCW learning also requires appropriate censoring assumptions and sufficient follow-up at the chosen horizon.

The package currently supports one right-censored outcome. It does not fit left- or interval-censored data, competing risks, left truncation, recurrent events, longitudinal updates, or cure models. Nested tuning is limited to stratified outer folds. Probability evaluation needs model-based curves, adequate censoring support, and a suitable reference model; the reported intervals exclude fitting and selection uncertainty. Interpretation is predictive, and restricted remaining-time summaries do not estimate an unobserved tail. Neural Cox handles ties without an external runtime, but retains proportional hazards and a fixed training budget with no guarantee of a global optimum.

The examples use one generated realization and historical data, not prospective observations. Time and memory results apply to the stated settings and hardware. Additive kernels can remain expensive with many distinct predictor values; parallel workers use more memory and need not be faster. Numerical accuracy does not establish statistical adequacy.

Installing the engines together simplifies use but requires more dependencies and compatibility checks for their versions. Further work includes repeated benchmarks, time and memory comparisons on other systems, additional neural architectures, and independent validation. BART and other outcome structures need their own estimation and prediction definitions. Within the present scope, **survlearning** provides a common R interface while preserving the statistical meaning of each prediction.

Acknowledgments

The author thanks Melanie Anaís García Fernández for preparing the preliminary presentation on survival learning.

Computational details

The package and replication code use GPL (version 3 or later); engines retain their licenses. Backblaze is credited as the data source, and its data-use conditions apply; the data are not relicensed as package source code.

References

Apley DW, Zhu J (2020). "Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models." *Journal of the Royal Statistical Society: Series B*, **82**(4), 1059–1086. doi:10.1111/rssb.12377.

Backblaze (2026). *Drive Stats: Hard Drive Reliability and Test Data*. 2013 historical data; accessed 2026-10-20, URL https://www.backblaze.com/cloud-storage/resources/hard-drive-test-data.

Barnwal A, Cho H, Hocking T (2022). "Survival Regression with Accelerated Failure Time Model in XGBoost." *Journal of Computational and Graphical Statistics*, **31**(4), 1292–1302. doi:10.1080/10618600.2022.2067548.

Biganzoli E, Boracchi P, Mariani L, Marubini E (1998). "Feed Forward Neural Networks for the Analysis of Censored Survival Data: A Partial Logistic Regression Approach." *Statistics in Medicine*, **17**(10), 1169–1186.

Binder H, Schumacher M (2008). "Allowing for Mandatory Covariates in Boosting Estimation of Sparse High-Dimensional Survival Models." *BMC Bioinformatics*, **9**, 14. doi:10.1186/1471-2105-9-14.

Breiman L, Friedman JH, Olshen RA, Stone CJ (1984). *Classification and Regression Trees*. Wadsworth.

Chen T, Guestrin C (2016). "XGBoost: A Scalable Tree Boosting System." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. doi:10.1145/2939672.2939785.

Chen T, He T, Benesty M, Khotilovich V, Tang Y, Cho H, Chen K, Mitchell R, Cano I, Zhou T, Li M, Xie J, Lin M, Geng Y, Li Y, Yuan J, Cortes D (2026). *xgboost: Extreme Gradient Boosting*. doi:10.32614/CRAN.package.xgboost. R package version 3.2.1.1, URL https://CRAN.R-project.org/package=xgboost.

Cox DR (1972). "Regression Models and Life-Tables." *Journal of the Royal Statistical Society B*, **34**(2), 187–220. doi:10.1111/j.2517-6161.1972.tb00899.x.

Efron B (1977). "The Efficiency of Cox's Likelihood Function for Censored Data." *Journal of the American Statistical Association*, **72**(359), 557–565. doi:10.1080/01621459.1977.10480613.

El Badisy I (2026). ***survalis**: Interpretable Survival Machine Learning Framework*. doi:10.32614/CRAN.package.survalis. R package version 1.0.0, URL https://CRAN.R-project.org/package=survalis.

Fouodo CJK, König IR, Weihs C, Ziegler A, Wright MN (2018). "Support Vector Machines for Survival Analysis with R." *The R Journal*, **10**(1), 412–423. doi:10.32614/RJ-2018-005.

Friedman J, Hastie T, Tibshirani R (2010). "Regularization Paths for Generalized Linear Models via Coordinate Descent." *Journal of Statistical Software*, **33**(1), 1–22. doi:10.18637/jss.v033.i01.

Friedman JH (2001). "Greedy Function Approximation: A Gradient Boosting Machine." *The Annals of Statistics*, **29**(5), 1189–1232. doi:10.1214/aos/1013203451.

Gerds TA, Kattan MW, Schumacher M, Yu C (2013). "Estimating a Time-Dependent Concordance Index for Survival Prediction Models with Covariate Dependent Censoring." *Statistics in Medicine*, **32**(13), 2173–2184. doi:10.1002/sim.5681.

Gerds TA, Schumacher M (2006). "Consistent Estimation of the Expected Brier Score in General Survival Models with Right-Censored Event Times." *Biometrical Journal*, **48**(6), 1029–1040. doi:10.1002/bimj.200610301.

Goldberg Y, Kosorok MR (2017). "Support Vector Regression for Right Censored Data." *Electronic Journal of Statistics*, **11**, 532–569. doi:10.1214/17-EJS1231.

Goulart PJ, Chen Y (2026). "Clarabel: An Interior-Point Solver for Conic Programs with Quadratic Objectives." *Mathematical Programming Computation*. doi:10.1007/s12532-026-00320-7.

Haider H (2026). *MTLR: Survival Prediction with Multi-Task Logistic Regression*. doi:10.32614/CRAN.package.MTLR. R package version 0.2.2, URL https://CRAN.R-project.org/package=MTLR.

Harrell FE, Califf RM, Pryor DB, Lee KL, Rosati RA (1982). "Evaluating the Yield of Medical Tests." *JAMA*, **247**(18), 2543–2546. doi:10.1001/jama.1982.03320430047030.

Harrell FE, Lee KL, Mark DB (1996). "Multivariable Prognostic Models: Issues in Developing Models, Evaluating Assumptions and Adequacy, and Measuring and Reducing Errors." *Statistics in Medicine*, **15**(4), 361–387.

Hester J, Henry L, Muller K, Ushey K, Wickham H, Chang W (2026). *withr: Run Code 'With' Temporarily Modified Global State*. doi:10.32614/CRAN.package.withr. R package version 3.0.3, URL https://CRAN.R-project.org/package=withr.

Hvitfeldt E, Frick H (2026). ***censored*: *parsnip* *Engines for Survival Models***. Accessed 2026-10-20, URL https://censored.tidymodels.org/.

Ishwaran H, Kogalur U (2026). *Fast Unified Random Forests for Survival, Regression, and Classification (RF-SRC)*. R package version 3.9.0, URL https://cran.r-project.org/package=randomForestSRC.

Ishwaran H, Kogalur UB, Blackstone EH, Lauer MS (2008). "Random Survival Forests." *The Annals of Applied Statistics*, **2**(3), 841–860. doi:10.1214/08-AOAS169.

Jackson CH (2016). "**flexsurv**: A Platform for Parametric Survival Modeling in R." *Journal of Statistical Software*, **70**(8), 1–33. doi:10.18637/jss.v070.i08.

Jaeger BC, Welden S, Lenoir K, Pajewski NM (2022). "aorsf: An R package for supervised learning using the oblique random survival forest." *Journal of Open Source Software*, **7**(77), 4705. URL https://doi.org/10.21105/joss.04705.

Jaeger BC, Welden S, Lenoir K, Speiser JL, Segar MW, Pandey A, Pajewski NM (2023). "Accelerated and interpretable oblique random survival forests." *Journal of Computational and Graphical Statistics*. URL https://doi.org/10.1080/10618600.2023.2231048.

Kalbfleisch JD, Prentice RL (2002). *The Statistical Analysis of Failure Time Data*. 2nd edition. John Wiley & Sons. doi:10.1002/9781118032985.

Kaplan EL, Meier P (1958). "Nonparametric Estimation from Incomplete Observations." *Journal of the American Statistical Association*, **53**(282), 457–481. doi:10.1080/01621459.1958.10501452.

Katzman JL, Shaham U, Cloninger A, Bates J, Jiang T, Kluger Y (2018). "DeepSurv: Personalized Treatment Recommender System Using a Cox Proportional Hazards Deep Neural Network." *BMC Medical Research Methodology*, **18**(1), 24. doi:10.1186/s12874-018-0482-1.

Kingma DP, Ba J (2015). "Adam: A Method for Stochastic Optimization." In *International Conference on Learning Representations*. URL https://arxiv.org/abs/1412.6980.

Kuhn M, Vaughan D (2026). *parsnip: A Common API to Modeling and Analysis Functions*. doi:10.32614/CRAN.package.parsnip. R package version 1.6.1, URL https://CRAN.R-project.org/package=parsnip.

Lawless JF (2003). *Statistical Models and Methods for Lifetime Data*. 2nd edition. John Wiley & Sons. doi:10.1002/9781118033005.

Lyu Y (2026). ***SuperSurv**: A Unified Framework for Machine Learning Ensembles in Survival Analysis*. doi:10.32614/CRAN.package.SuperSurv. R package version 0.1.9, URL https://CRAN.R-project.org/package=SuperSurv.

Molstad AJ, Suder PM (2025). *penAFT: Fit the Semiparametric Accelerated Failure Time Model with Elastic Net and Sparse Group Lasso Penalties*. doi:10.32614/CRAN.package.penAFT. R package version 0.3.2, URL https://CRAN.R-project.org/package=penAFT.

Pölsterl S (2020). "**scikit-survival**: A Library for Time-to-Event Analysis Built on Top of **scikit-learn**." *Journal of Machine Learning Research*, **21**(212), 1–6. URL https://jmlr.org/papers/v21/20-729.html.

Pölsterl S, Navab N, Katouzian A (2015). "Fast Training of Support Vector Machines for Survival Analysis." In *Machine Learning and Knowledge Discovery in Databases*, pp. 243–259. Springer-Verlag. doi:10.1007/978-3-319-23525-7_15.

R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. doi:10.32614/R.manuals. URL https://www.R-project.org/.

Ridgeway G, Developers G (2026). *gbm: Generalized Boosted Regression Models*. doi:10.32614/CRAN.package.gbm. R package version 2.3.1, URL https://CRAN.R-project.org/package=gbm.

Ripley BD, Venables WN (2002). *Modern Applied Statistics with S*. Fourth edition. Springer, New York. ISBN 0-387-95457-0, URL https://www.stats.ox.ac.uk/pub/MASS4/.

Shamsutdinova D, Stahl D (2025). *survcompare: Nested Cross-Validation to Compare Cox-PH, Cox-Lasso, Survival Random Forests*. doi:10.32614/CRAN.package.survcompare. R package version 0.3.0, URL https://CRAN.R-project.org/package=survcompare.

Simon N, Friedman J, Hastie T, Tibshirani R (2011). "Regularization Paths for Cox's Proportional Hazards Model via Coordinate Descent." *Journal of Statistical Software*, **39**(5), 1–13. doi:10.18637/jss.v039.i05.

Sonabend R, Király FJ, Bender A, Bischl B, Lang M (2021). "**mlr3proba**: An R Package for Machine Learning in Survival Analysis." *Bioinformatics*, **37**(17), 2789–2791. doi:10.1093/bioinformatics/btab039.

Suder PM, Molstad AJ (2022). "Scalable Algorithms for Semiparametric Accelerated Failure Time Models in High Dimensions." *Statistics in Medicine*, **41**(6), 933–949. doi:10.1002/sim.9264.

SuperSurv Authors (2026). "Ensemble vs. Best Model Selection." Package vignette, accessed 2026-10-21, URL https://cran.r-project.org/web/packages/SuperSurv/vignettes/supersurv-best.html.

Therneau T, Atkinson B (2026). *rpart: Recursive Partitioning and Regression Trees*. doi:10.32614/CRAN.package.rpart. R package version 4.1.27, URL https://CRAN.R-project.org/package=rpart.

Therneau TM (2026). *A Package for Survival Analysis in R*. R package version 3.8-12, URL https://CRAN.R-project.org/package=survival.

Therneau TM, Grambsch PM (2000). *Modeling Survival Data: Extending the Cox Model*. Springer-Verlag, New York. doi:10.1007/978-1-4757-3294-8.

Van Belle V, Pelckmans K, Van Huffel S, Suykens JAK (2011). "Support Vector Methods for Survival Analysis: A Comparison between Ranking and Regression Approaches." *Artificial Intelligence in Medicine*, **53**, 107–118. doi:10.1016/j.artmed.2011.06.006.

Wey A, Connett J, Rudser K (2015). "Combining Parametric, Semi-parametric, and Non-parametric Survival Models with Stacked Survival Models." *Biostatistics*, **16**(3), 537–549. doi:10.1093/biostatistics/kxv001.

Wright MN, Ziegler A (2017). "ranger: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R." *Journal of Statistical Software*, **77**(1), 1–17. doi:10.18637/jss.v077.i01.

Yang X, Abraham L, Kim S, Smirnov P, Ruan F, Haibe-Kains B, Tibshirani R (2022). "FastCPH: Efficient Survival Analysis for Neural Networks." doi:10.48550/arXiv.2208.09793. ArXiv:2208.09793.

Yu CN, Greiner R, Lin HC, Baracos V (2011). "Learning Patient-Specific Cancer Survival Distributions as a Sequence of Dependent Regressors." In *Advances in Neural Information Processing Systems*, volume 24. URL https://www.cs.cornell.edu/~cnyu/papers/nips11_survival.pdf.

Zou H, Hastie T (2005). "Regularization and Variable Selection via the Elastic Net." *Journal of the Royal Statistical Society B*, **67**(2), 301–320. doi:10.1111/j.1467-9868.2005.00503.x.

A. Exact censored SVM computation

A.1. Dual derivation and positive semidefiniteness

Introduce slacks $\xi_i \geq 0$ for $Y_i - g_i \leq \xi_i$ and $\xi_i^* \geq 0$ for $g_i - Y_i \leq \xi_i^*$ when $\delta_i = 1$. Their multipliers satisfy $0 \leq \alpha_i, \beta_i \leq C$. Differentiating the Lagrangian with respect to w and b gives

$$w = \sum_i (\alpha_i - \delta_i \beta_i)\phi(x_i), \qquad \sum_i (\alpha_i - \delta_i \beta_i) = 0.$$

Thus $c_i = \alpha_i - \delta_i \beta_i$ satisfies $-C\delta_i \leq c_i \leq C$. Eliminating the primal variables gives Equation 35, with Hessian in the two-block coordinates

$$H = \begin{pmatrix} K & -KD \\ -DK & DKD \end{pmatrix} = [I, -D]^\top K [I, -D]. \tag{57}$$

Thus $v^\top H v \geq 0$ for every v when K is positive semidefinite; no positive-definite projection is needed.

A.2. Sparse additive representation

For one predictor normalized to $[0, 1]$, $k(u, v) = 1 - u - v + 2\min(u, v)$. The constant and linear terms vanish in $c^\top K c$ when $\mathbf{1}^\top c = 0$. For the remaining integrated-step kernel, the fitted function has constant derivative between successive observed coordinates. When its values at training coordinates are fixed, linear interpolation minimizes the integral of the squared derivative. Retaining all coordinates $u_0 < \cdots < u_m$ gives

$$\frac{1}{2}\|g\|_{\mathcal{H}}^2 = \frac{1}{4}\int_0^1 \{g'(u)\}^2 \, du = \frac{1}{4}\sum_{k=1}^{m} \frac{(\theta_k - \theta_{k-1})^2}{u_k - u_{k-1}}.$$

A constant is absorbed in the unpenalized intercept, so $\theta_0 = 0$ identifies the component without restricting fitted values. Summing over predictors gives Equation 36.

Let Z record the corresponding component value for each observation, and let Q be the block-tridiagonal penalty matrix, so the objective is

$$\tfrac{1}{2}\theta^\top Q \theta + C\left(\sum_i \xi_i + \sum_{i:\delta_i=1} \xi_i^*\right)$$

subject to $Y - Z\theta - b\mathbf{1} \leq \xi$, $(Z\theta + b\mathbf{1} - Y)_{\delta=1} \leq \xi^*$, and nonnegative slacks. The design contains at most np nonzero entries, and $m = \sum_j m_j \leq p(n-1)$ component coefficients are retained. Factorization may require additional storage and fill-in, so linear running time is not guaranteed. Duplicate observations keep separate loss constraints even when they share component coordinates.

For a new observation, interpolation uses stored training coordinates. Outside a training range, the fitted function is constant in that coordinate because $\sum_i c_i = 0$. Ranges are not recomputed from new observations or from only those training observations with nonzero dual coefficients.

A.3. Numerical certificates, intercepts and ties

Diagonal preconditioning and response centering do not change the problem. Clarabel and OSQP must report full convergence. The package independently reconstructs c from the multipliers and checks equality and box constraints, primal feasibility, and the objective gap. With $h = Z^\top c$, the dual value is

$$D(c) = Y^\top c - \tfrac{1}{2} h^\top Q^{-1} h.$$

The accepted relative gap is

$$\frac{|P(\widehat{\theta}, \widehat{b}) - D(\widehat{c})|}{\max\{1, |P(\widehat{\theta}, \widehat{b})|, |D(\widehat{c})|\}} \leq 10^{-5},$$

in addition to feasibility checks. Application gaps are substantially smaller. The default solver tolerance is 10^{-12}, and diagnostic values are stored.

For fixed θ, let $r_i = Y_i - (Z\theta)_i$. The intercept-loss derivative starts at $-n$ and jumps by $1 + \delta_i$ at r_i. The weighted order statistic at cumulative weight n determines the minimizing point or interval; its midpoint fixes a nonunique intercept.

The grid has training-fixed spacing $10^{-8} \max_i Y_i$ and center $\max_i Y_i$, preventing small numerical differences from creating apparent concordance. Raw scores use `prediction_tolerance = 0`; no tolerance uses test outcomes. Cost candidates share the design, penalty, and constraints within each training fold, but start the solver afresh. The selected model is refitted on all training rows.

Affiliation:

Pedro Luiz Ramos
Departamento de Estadística, Facultad de Matemáticas
Pontificia Universidad Católica de Chile
Santiago, Chile
E-mail: pedro.ramos@uc.cl